

Package ‘timesift’

October 2, 2026

Title Learn Predictive Representations of Time-Varying Data

Version 0.3.1

Description Fits and compares representations of time-varying data against a prediction target. Given a table of targets and a table of time-stamped series belonging to them, it builds each candidate representation, from the record unreduced through a calendar grain such as a week or a month to a lookback anchored on each target, fits the requested learners on each, scores every candidate on one set of held-out folds, and stacks the out-of-fold predictions into an ensemble. Calendar-aware binning keeps a bin a real week or month rather than a fixed block of hours. Learners, response heads and metrics are registered rather than hard-coded, so adding one is a registration and not a fork of the fitting code. The penalised baseline is an elastic net fitted by cyclic coordinate descent along a warm-started path, following Friedman, Hastie and Tibshirani (2010) <doi:10.18637/jss.v033.i01>. The shipped default is presence-absence with a joint multi-label head scored by the true skill statistic of Allouche, Tsoar and Kadmon (2006) <doi:10.1111/j.1365-2664.2006.01214.x>, the setting used for species distribution modelling from microclimate loggers.

License MIT + file LICENSE

Encoding UTF-8

Language en-GB

URL <https://gillescolling.com/timesift/>,
<https://github.com/gcol33/timesift>

BugReports <https://github.com/gcol33/timesift/issues>

Depends R (>= 4.1)

LinkingTo cpp11

Imports grDevices, graphics, rlang, stats, tidyselect (>= 1.2.0),
tools, utils

Suggests testthat (>= 3.0.0), torch, glmnet, ranger, lme4, lmerTest,
emmeans, knitr, rmarkdown, spelling, withr

VignetteBuilder knitr

Config/testthat/edition 3

Config/roxygen2/version 8.1.0

NeedsCompilation yes

Author Gilles Colling [aut, cre, cph] (ORCID:
<<https://orcid.org/0000-0003-3070-6066>>)

Maintainer Gilles Colling <gilles.colling051@gmail.com>

Repository CRAN

Date/Publication 2026-10-02 10:40:02 UTC

Contents

artifacts	3
average_precision	4
bind_channels	5
build_representation	6
calendar_channels	6
combine	7
coverage	8
cv	9
digest_array	10
elasticnet	11
ensemble	13
ensemble_combine	14
ensemble_fit	15
ensemble_weights	16
feature_matrix	16
fit_learner	17
fold_map	18
forest	19
grains	20
grain_contrasts	21
grain_ladder	22
grain_matrix	24
implied_skill	27
kappa_score	29
learner	30
lookback_matrix	31
native	34
occlusion	35
paired_contrast	37
plot.timesift	39
plot.timesift_ladder	39
plot.timesift_selection	40
positive_weights	41
predict.timesift	42
register_learner	43

register_metric	43
register_response	44
roc_auc	45
scorable_cells	46
score_predictions	47
select_grain	48
simulate_records	52
stepwise	55
timesift	56
timesift_report	59
timesift_set	60
torch_learners	61
train_control	62
tss	64
tss_inflation	65

Index	67
--------------	-----------

artifacts	<i>Read and write the artifacts that cross the language boundary</i>
-----------	--

Description

The representation is built twice, once per language, from one shared core. The response matrix, the fold map and the mask of scorable cells are not: they are built once and read wherever they are needed, because a fold map drawn from a seed in R and one drawn from the same seed in Python are different maps, and aligning the two random streams would be the wrong fix.

Usage

```
write_folds(x, file)

read_folds(file, units = NULL)

write_response(y, file)

read_response(file, units = NULL)

write_cells(cells, file)

read_cells(file)
```

Arguments

x	A fold map from <code>fold_map()</code> , or any named integer vector.
file	Path to write to or read from.

units	The units to align to, in the order they are wanted. A unit the file has no row for is an error; a unit the file carries beyond these is dropped. NULL, the default, returns the file's own rows in the file's own order.
y	A response matrix, as <code>scorable_cells()</code> takes one.
cells	A mask from <code>scorable_cells()</code> .

Details

These six functions are the handover. The Python side carries `write_folds()` and its siblings under the same names; they write the same bytes from the same artifact and read the same files, so a fold map built in either language is usable in the other without the caller knowing the format.

The format is normative and is given in `inst/spec/representation.md`: CSV, UTF-8, a header row, no quoting, LF line endings on every platform, numbers at twelve significant digits, and rows ordered by the identifier under C collation.

Value

The writers return file, invisibly. `read_folds()` returns a named integer vector of class `timesift_folds`, `read_response()` a numeric matrix with the units in its row names, and `read_cells()` a `timesift_cells` data frame.

Examples

```
set.seed(1)
y <- matrix(rbinom(120, 1, 0.3), nrow = 20,
            dimnames = list(sprintf("p%02d", 1:20), paste0("sp", 1:6)))
f <- fold_map(y, v = 4)

path <- tempfile(fileext = ".csv")
write_folds(f, path)
identical(read_folds(path, names(f)), f[names(f)])
unlink(path)
```

average_precision	<i>Average precision</i>
-------------------	--------------------------

Description

The area under the precision-recall curve, as the step sum over the distinct predictions: at each, the precision of calling every unit at or above it a presence, weighted by the share of presences that cut adds. Units sharing a prediction enter together, so the order they arrived in does not move the score. Its floor is the prevalence rather than one half, which makes it the reading of how well presences are ranked above absences where presences are rare, and it is reported beside `roc_auc()` for that reason.

Usage

```
average_precision(y, p)
```

Arguments

y Observed presence-absence, 0/1 or logical.
p Predicted scores for the same units, in the same order. Higher means presence.

Value

One number, or NA where the cell defines none.

Examples

```
average_precision(c(0, 0, 1, 1), c(0.1, 0.2, 0.8, 0.9))
average_precision(c(0, 1, 0, 1), c(0.1, 0.2, 0.8, 0.9))
```

bind_channels	<i>Put channels side by side</i>
---------------	----------------------------------

Description

Joins representations of the same units and bins into one array, in the order given. It is how a temperature reading, an external product such as snow cover, and the calendar position of each bin reach a model as one input.

Usage

```
bind_channels(...)
```

Arguments

... Two or more representations of shape [unit, bin, channel], agreeing on their units and bins.

Value

One array carrying every channel, in the order the arguments are given and, inside each argument, in its own channel order. The attributes are the first argument's, with stats the joined names, except static and position, which name channels and so name those of every argument.

Examples

```
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 60)
d <- data.frame(plot = rep(c("a", "b"), each = length(t)), t = rep(t, 2),
               temp = rnorm(2 * length(t)))
x <- grain_matrix(d, plot, t, temp, grain = "week")
dimnames(bind_channels(x, calendar_channels(x)))[[3]]
```

build_representation	<i>Build one representation for a set of targets</i>
----------------------	--

Description

Turns a representation and the two tables into the `[target, bin, channel]` array a learner is fitted on. It is the one place the fitting layer builds an array, so `timesift()` and `predict.timesift()` reach a record the same way and a candidate refitted on new targets is built from the settings its own arm was.

Usage

```
build_representation(rep, series, targets, spec)
```

Arguments

rep	A representation from <code>native()</code> , <code>grain()</code> , <code>multigrain()</code> or <code>lookback()</code> .
series	The long table of readings, or NULL where the targets carry the whole predictor block.
targets	The table of prediction targets.
spec	The resolved settings <code>timesift()</code> carries: the identifier, time, value, anchor and static columns it settled on.

Details

The rows are the targets, in the order the fitting layer keeps them: sorted by identifier where one target row belongs to each unit, and in the targets' own order where `target_time` anchors them. Columns named in `static` are appended as channels holding one value per target, constant across the bins.

Value

A `timesift_matrix`.

calendar_channels	<i>Where in the year, or the day, each bin sits</i>
-------------------	---

Description

An encoder that ends in global pooling discards when a thermal event happened, so the position of a bin in the year has to be given to it as input if it is to be used at all. These channels carry that position as the sine and cosine of the bin's fractional place in the year, which is continuous across the turn of the year where the fraction itself is not. On a record read finer than a day, the same pair for the place in the day carries where a reading sits in the daily cycle.

Usage

```
calendar_channels(x, cycles = "year")
```

Arguments

<code>x</code>	A <code>grain_matrix()</code> result. A <code>lookback_matrix()</code> result has no place in the calendar and is refused.
<code>cycles</code>	Which cycles to place each bin in, "year", "day" or both, in the order given. The day cycle reads bins that sit less than a day apart; at a day or coarser every bin would sit at the same place in the day, and it is refused.

Details

They are the time index of each bin, not a summary of the readings, so adding them introduces no hand-built thermal feature: whatever a model does with them it could have done with a calendar.

The position is read at the midpoint of the record each bin holds, in UTC, so a bin the record only partly covers sits at the phase it was actually measured over. A site's longitude and a zone's offset move the day's phase by the same amount for every bin, which a model absorbs, where a local clock's summer time would move it twice a year. `inst/spec/representation.md` is the normative description.

Value

An array of the same units and bins with two channels per cycle, `year_sin` and `year_cos`, `day_sin` and `day_cos`, identical across units. Combine it with the readings using `bind_channels()`. The channels are recorded as positions in the position attribute, and the encoders of `torch_learners` read them at their own amplitude rather than standardise them.

Examples

```
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 400)
d <- data.frame(plot = "a", t = t, temp = sin(seq_along(t) / 500))
x <- grain_matrix(d, plot, t, temp, grain = "month")
round(calendar_channels(x)[1, 1:4, ], 3)

hourly <- grain_matrix(d, plot, t, temp, grain = "native")
round(calendar_channels(hourly, cycles = c("year", "day"))[1, 1:4, ], 3)
```

Description

`c()` on learners is the set of them, and on representations the set of those. A set handed to `c()` again splices, so a set can be added to rather than rewritten, which is what `list()` cannot do: `c(base, cnn())` where `base` is already a set.

Usage

```
## S3 method for class 'timesift_learner'
c(...)

## S3 method for class 'timesift_models'
c(...)

## S3 method for class 'timesift_representation'
c(...)

## S3 method for class 'timesift_sift'
c(...)
```

Arguments

... Learners, representations, or sets of either.

Details

models and sift take either form. A length-one string is the name of a registered learner.

Value

A timesift_models for learners and a timesift_sift for representations.

Examples

```
base <- c(elasticnet(), forest())
base
c(base, stepwise())

c(grains("day", "week"), lookback("30 days"))
```

coverage

Which units reach which bins

Description

A representation needs every unit in every bin, and `grain_matrix()` refuses a record where one is missing rather than pad it. This is the same binning laid out so the gaps can be read: how many readings each unit has in each bin, over every bin the calendar tiles the record with from the first bin any unit touches to the last. A logger that started late, stopped early or lost a month is a row with zeros in it; a bin the whole record skips is a column of zeros.

Usage

```
coverage(data, id, time, grain = "day", year_start = "09-01")
```

Arguments

<code>data</code>	A data frame of readings in long form, one row per reading.
<code>id</code>	Column identifying the unit carrying the sensor. A bare column name or a string.
<code>time</code>	Column of reading instants, POSIXct. A bare column name or a string.
<code>grain</code>	One of "native", "halfday", "day", "week", "month", "season", "year". The four coarse grains follow the calendar, so a bin is a real week or month rather than a fixed block of hours. Naming several grains returns one representation per grain, a <code>timesift_set()</code> . A function is called on the reading instants and must return the POSIXct start of each reading's bin, which is how a calendar the package does not carry, such as astronomical seasons, is binned.
<code>year_start</code>	"MM-DD" boundary of the hydrological year, used by "season" and "year". Defaults to "09-01".

Details

What to do about a gap is the analyst's decision, and this is the table it is made on: drop the units that do not span the record, cut the record to the span every unit covers, or move to a grain the gap does not reach. Nothing here fills a cell.

Value

An integer matrix of reading counts, one row per unit and one column per bin, of class `timesift_coverage`, with the units and the ISO-8601 bin starts as `dimnames` and the `grain` and the `bin_start` instants as attributes.

Examples

```
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 40)
d <- data.frame(plot = rep(c("a", "b"), each = length(t)), t = rep(t, 2),
               temp = rnorm(2 * length(t)))
# Unit b loses the calendar week beginning Monday 6 September.
lost <- d$plot == "b" & d$t >= as.POSIXct("2021-09-06", tz = "UTC") &
  d$t < as.POSIXct("2021-09-13", tz = "UTC")
coverage(d[!lost, ], plot, t, grain = "week")
```

Description

The resampling `timesift()` scores on. `cv()` deals units into folds balanced on the stratifying value; `grouped_cv()` deals whole groups, keeping every target sharing a group value on one side of each split.

Usage

```
cv(v = 10L, seed = 1L, strata = 5L)

grouped_cv(group, v = 10L, seed = 1L)
```

Arguments

v	Number of folds.
seed	Random seed, fixed so the map is reproducible.
strata	Number of strata, or 1 for no stratification.
group	The grouping: the name of a column of targets, or a vector with one value per target.

Details

resampling also accepts a fold vector or a `fold_map()` result directly, which is how a split the package has no constructor for – a spatial block, a season held out whole – reaches the same fitting path.

Value

A `timesift_resampling`.

Examples

```
cv(v = 5L)
grouped_cv("site")
```

digest_array

The cross-language digest of a representation

Description

The MD5 the contract in `inst/spec/representation.md` defines, and what the fixtures in `inst/spec/fixtures/` are checked against. `timesift.digest_array()` on the Python side is the same function: the same array gives the same string in either language, which is what makes a digest a statement about the representation rather than about the machine.

Usage

```
digest_array(x)
```

Arguments

x	A <code>grain_matrix()</code> result, or any numeric array.
---	---

Details

Values are traversed in the array's own order, unit fastest, then bin, then channel; each is formatted to twelve decimal places, joined with a line feed and terminated with one, and the UTF-8 bytes of that are hashed. The line ending is a line feed on every platform. It is written explicitly because `writeln()` emits CRLF on Windows, which would make a digest depend on the machine that produced it.

The array has to be finite. `%.12f` writes an infinity as `Inf` here and as `inf` in Python, and R tells a missing value apart from a not-a-number where Python has one spelling for both, so a digest over such an array would say something about the language rather than about the representation. A representation never holds one: a reading that is not a finite number is refused where the record is read.

Value

The digest, a single string of 32 hexadecimal characters.

Examples

```
digest_array(array(c(1, -0.5), dim = c(2L, 1L, 1L)))
```

elasticnet

Penalised regression on the flattened representation

Description

One elastic net per variable, over every bin-by-channel column of the representation and, by default, their squares. There is no discrete selection step: the penalty path uses every column and shrinks, and the penalty itself is chosen by an inner cross-validation on the fitting units, so nothing about the model is decided outside the fold it is fitted in.

Usage

```
elasticnet(
  data = NULL,
  alpha = 0.5,
  n_inner = 5L,
  squares = TRUE,
  s = "lambda.min",
  n_lambda = 100L,
  thresh = 1e-08,
  threads = 1L,
  seed = 1L
)
```

Arguments

data	A representation the learner is pinned to, or NULL to run across every representation of the run.
alpha	Elastic-net mixing, 1 lasso and 0 ridge.
n_inner	Folds of the inner cross-validation that chooses the penalty.
squares	Add the square of every column, giving the same quadratic capacity a second-order polynomial term would.
s	Which penalty of the inner path to predict at: "lambda.min", "lambda.1se", or a penalty of its own, which is interpolated between the two points of the path around it.
n_lambda	Points of the penalty path.
thresh	Where the coordinate descent stops, read off the largest coefficient move of a pass. The default leaves the fit as close to the optimum as glmnet's own default does; a looser one is faster and a tighter one costs time roughly in proportion.
threads	How many fits of one response's inner cross-validation run at once. The path on every fitting unit and the path of each inner fold are one independent fit each, so they parallelise without sharing anything, and n_inner + 1 threads is as many as a response can use. The default is serial, because a package does not take a machine's cores without being asked. What comes back does not depend on it.
seed	Seed for the inner cross-validation's fold draw, which is random and would otherwise make the fit irreproducible.

Details

The family is the response head's: a binary cross-entropy loss fits a logistic model and a squared-error loss a linear one, so the learner is the same under a presence-absence head and under a continuous one. So are the case weights: the head's weights, `positive_weights()` for presence-absence, are what every learner that ships fits under.

The inner folds are dealt for each response and stratified on it, so a rare outcome is spread over them as evenly as its count allows. A presence-absence response whose inner training sets cannot each hold two of each outcome, the fewest a logistic path is fitted to, has too few of one outcome to choose a penalty on. It is predicted its share among the fitting units, as a response holding one outcome is, and the fit names every such response in `unfitted`.

A reweighted fit that does not settle at a penalty ends its path there and keeps the points before it, which is what glmnet does with the same event, and the penalty is chosen over the points fitted. It happens where a rare outcome is nearly separable at the small end of the path. The fit names every response whose path on the fitting units, or on any inner fold, ended that way in `stopped`.

This is the aggregate-feature side of the comparison the package was built for, and it is the fair opponent for a network: a per-fold discrete selector pays selection variance a network never pays, so beating that one is not a matched result.

The path is fitted by the same core the Python package calls, so the two return the same coefficients for the same input. Its conventions are glmnet's, which is what the arm is measured against: weights normalised to sum to one, columns centred and scaled by their weighted mean and weighted standard deviation, a hundred penalties down from the smallest that leaves every coefficient at zero, and the held-out deviance read fold by fold.

Value

A `learner()`.

Examples

```
elasticnet(alpha = 0.5)
```

ensemble	<i>How the candidates are combined</i>
----------	--

Description

Every candidate emits an out-of-fold prediction for every scorable cell over the same folds, so the combination is arithmetic on those predictions and nothing else. `ensemble()` says which arithmetic.

Usage

```
ensemble(
  method = c("stack", "mean", "median", "weighted"),
  scope = c("all", "learners", "representations"),
  metric = NULL,
  response = NULL
)
```

Arguments

method	How the members are combined.
scope	Which candidates are eligible.
metric	Name of the registered metric the eligibility and the "weighted" weights are read by, or NULL for the score the run already carries.
response	Name of the registered response head whose loss "stack" minimises, or NULL for the head the run was fitted under. Naming a head the run does not fit toward is an error rather than an override, and <code>ensemble_fit()</code> called on its own reads NULL as "presence_absence".

Details

"stack" fits non-negative weights summing to one on the out-of-fold predictions alone, never on in-sample ones, minimising the response head's loss over the scorable cells: binomial deviance for presence-absence. One weight vector covers every response, because per-response weights would be fitted on the handful of cells a rare response has. "mean" and "median" combine without fitting anything. "weighted" takes each candidate's own mean score, keeps its non-negative part and rescales those to sum to one, so a candidate scoring at or below zero is left out and the rest are weighted by how well they scored.

scope says which candidates are eligible. "all" is every candidate. "learners" keeps the several learners that read the representation of the best-scoring candidate, and "representations" keeps the one learner of the best-scoring candidate across the representations it ran on; both are read off the same mean scores the report shows.

Value

A `timesift_ensemble`.

Examples

```
ensemble()
ensemble("weighted", scope = "learners")
```

<code>ensemble_combine</code>	<i>Combine one prediction per member into one prediction</i>
-------------------------------	--

Description

Combine one prediction per member into one prediction

Usage

```
ensemble_combine(stack, preds)
```

Arguments

<code>stack</code>	A <code>ensemble_fit()</code> result.
<code>preds</code>	Named list of [target, response] matrices, one per member of the stack.

Value

One [target, response] matrix.

Examples

```
set.seed(1)
y <- matrix(rbinom(200, 1, 0.4), nrow = 50,
            dimnames = list(sprintf("p%02d", 1:50), paste0("sp", 1:4)))
folds <- fold_map(y, v = 5)
truth <- matrix(runif(200), nrow = 50, dimnames = dimnames(y))
oof <- list(good = 0.8 * y + 0.2 * truth, noise = truth)
st <- ensemble_fit(oof, y, scorable_cells(y, folds), folds)
dim(ensemble_combine(st, oof))
```

ensemble_fit

*Fit the combiner on the out-of-fold predictions***Description**

The combiner sees the out-of-fold predictions, the response, the mask of scorable cells and the fold map, and never a model, so it cannot read anything a candidate fitted in-sample.

Usage

```
ensemble_fit(oof, y, cells, folds, spec = ensemble(), scores = NULL)
```

Arguments

oof	Named list of [target, response] matrices, one per candidate.
y	The response matrix.
cells	The scorable-cell mask from scorable_cells() .
folds	The fold map.
spec	An ensemble() specification.
scores	The per-cell scores of the run, a data frame carrying candidate, variable, fold, score and scorable. Read only where spec names no metric of its own.

Details

The weights are fitted to the response on those predictions, so the combined prediction scored against the same response is scored on the data its weights were fitted to, and that score is optimistic. [timesift\(\)](#) evaluates the stack the other way: each outer fold's weights are fitted on inner out-of-fold predictions of its training targets and applied to the outer test fold.

Value

A timesift_stack: method, the named weights, and what they were fitted on.

Examples

```
set.seed(1)
y <- matrix(rbinom(200, 1, 0.4), nrow = 50,
            dimnames = list(sprintf("p%02d", 1:50), paste0("sp", 1:4)))
folds <- fold_map(y, v = 5)
truth <- matrix(runif(200), nrow = 50, dimnames = dimnames(y))
oof <- list(good = 0.8 * y + 0.2 * truth, noise = truth)
ensemble_fit(oof, y, scorable_cells(y, folds), folds)
```

ensemble_weights	<i>The weights the combiner fitted</i>
------------------	--

Description

The weights the combiner fitted

Usage

```
ensemble_weights(fit)
```

Arguments

fit	A timesift result, or the stack itself.
-----	---

Value

A named numeric vector, or NULL where the run fitted no combiner.

Examples

```
set.seed(1)
y <- matrix(rbinom(200, 1, 0.4), nrow = 50,
            dimnames = list(sprintf("p%02d", 1:50), paste0("sp", 1:4)))
folds <- fold_map(y, v = 5)
truth <- matrix(runif(200), nrow = 50, dimnames = dimnames(y))
oof <- list(good = 0.8 * y + 0.2 * truth, noise = truth)
ensemble_weights(ensemble_fit(oof, y, scorable_cells(y, folds), folds))
```

feature_matrix	<i>Bring an already-reduced feature table into a ladder</i>
----------------	---

Description

A representation the package did not build, such as a published set of hand-aggregated climate summaries, enters here. It becomes a one-channel [unit, feature, 1] array, which is what a learner reads, so a feature table and a temporal grain can be arms of the same [grain_ladder\(\)](#) and be scored on the same cells by the same rule.

Usage

```
feature_matrix(m, label = "features")
```

Arguments

m	A matrix or data frame of units by features, with unit identifiers in the row names or in a leading character or factor column.
label	The name the arm is reported under.

Details

It carries no time axis, because it has none: the reduction already happened, elsewhere, and what reaches the model is a list of numbers per unit. That is the whole point of comparing against it.

Value

A `timesift_matrix` of shape [unit, feature, 1].

Examples

```
m <- matrix(rnorm(30), nrow = 10,
            dimnames = list(sprintf("p%02d", 1:10), paste0("bio", 1:3)))
feature_matrix(m)
```

fit_learner	<i>Fit one learner at one grain</i>
-------------	-------------------------------------

Description

Fit one learner at one grain

Usage

```
fit_learner(
  learner,
  x,
  y,
  response = "presence_absence",
  control = NULL,
  group = NULL,
  ...
)

## S3 method for class 'timesift_fit'
predict(object, newdata, ...)
```

Arguments

learner	A <code>learner()</code> , or the name of a registered one.
x	A <code>grain_matrix()</code> result.
y	The response for the same units.
response	Name of the registered response head. "presence_absence" ships.
control	The run's <code>train_control()</code> . The learner's own control overrides it on the settings that control names, and a setting given in ... overrides both.
group	One value per unit of x naming the group it belongs to, or NULL. A learner whose fit declares group draws its inner splits by it. Under <code>grouped_cv()</code> the run hands every fit the grouping its fold map carries.
...	Passed to the learner's fit.
object	A <code>timesift_fit</code> .
newdata	A representation of the same channels for the units to predict.

Value

A `timesift_fit`, which `stats::predict()` takes a new representation.

Examples

```
set.seed(1)
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 120)
units <- sprintf("p%02d", 1:40)
d <- data.frame(plot = rep(units, each = length(t)), t = rep(t, length(units)),
                temp = as.numeric(replicate(length(units), rnorm(length(t)))))
x <- grain_matrix(d, plot, t, temp, grain = "month")
y <- matrix(rbinom(80, 1, 0.4), nrow = 40, dimnames = list(units, c("sp1", "sp2")))
fit <- fit_learner(elasticnet(), x, y)
dim(stats::predict(fit, x))
```

fold_map

Assign units to cross-validation folds

Description

One fold map, built once and read by everything that scores. Every learner in a ladder is then fitted and scored on identical splits, which is what makes the comparison between them paired rather than a comparison of two clouds of numbers.

Usage

```
fold_map(y, v = 10L, seed = 1L, strata = 5L, by = NULL, group = NULL)
```

Arguments

<code>y</code>	The response: a matrix or data frame of units by variables, with unit identifiers in the row names or in a leading character or factor column.
<code>v</code>	Number of folds.
<code>seed</code>	Random seed, fixed so the map is reproducible.
<code>strata</code>	Number of strata, or 1 for no stratification.
<code>by</code>	A numeric vector of length <code>nrow(y)</code> to stratify on instead of richness.
<code>group</code>	A vector of length <code>nrow(y)</code> . Rows sharing a value land in one fold, which is what repeated targets on one unit through time need: a unit split across folds would put the same unit on both sides of a split and the held-out score would be read on rows the model had already seen a near-copy of. NULL, the default, is every row on its own.

Details

Units are held out singly. Where the input a model reads is measured at the unit itself, as a logger in each plot is, a held-out unit brings its own measured input with it and nothing of its neighbours' reaches the model.

Folds are balanced within strata: units are grouped into `strata` equal-count groups of the stratifying value, shuffled inside each group, and dealt round-robin by one counter that runs on from each stratum into the next, so each fold carries the same mix and the folds are equal in size to within one unit whatever `v` is, up to one unit per fold. With a multi-variable response the default stratifies on richness, the number of variables present at a unit, because one fold map has to serve every variable at once and cannot be stratified on any single one of them.

Value

An integer vector of fold numbers named by unit, of class `timesift_folds`. Any named integer vector of the same shape is accepted wherever this is.

Examples

```
y <- matrix(rbinom(300, 1, 0.3), nrow = 60,
            dimnames = list(sprintf("p%02d", 1:60), paste0("sp", 1:5)))
f <- fold_map(y, v = 5)
table(f)
```

forest

Random forest on the flattened representation

Description

One forest per response, over every bin-by-channel column of the representation: a probability forest under a presence-absence head and a regression forest under a head with a squared-error loss. Trees split on one column at a time and pay nothing for columns that carry nothing, so a forest reads a wide tabular representation without a penalty path and without a selection step, and it finds an interaction between two bins that a linear model would need the product term for.

Usage

```
forest(data = NULL, trees = 500L, mtry = NULL, min_node = 1L, seed = 1L)
```

Arguments

data	A representation the learner is pinned to, or NULL to run across every representation of the run.
trees	Trees in the forest.
mtry	Columns tried at each split, or NULL for the square root of the column count.
min_node	Smallest node a split is made on.
seed	Seed for the bootstrap draw and the split sampling, which are random and would otherwise make the fit irreproducible.

Details

The case weights are the response head's, `positive_weights()` under presence-absence, and weight the bootstrap draw: a rare response is not fitted away here for a reason the other learners do not share, because every learner that ships reads the same weights.

Value

A `learner()`.

Examples

```
forest(trees = 200L)
```

grains

Several representations to run the same learners across

Description

The set a learner without a data = of its own is fitted at every member of. `grains()` names calendar grains, `lookbacks()` names lookback spans, and either takes its arguments as separate names or as one vector.

Usage

```
grains(..., stats = "mean", year_start = "09-01")
```

```
lookbacks(..., lag = "0 days", bins = 1L, stats = "mean")
```

```
as_sift(x)
```

Arguments

<code>...</code>	Grain names for <code>grains()</code> , <code>lookback</code> spans for <code>lookbacks()</code> , given as separate arguments or as one vector.
<code>stats</code>	Statistics computed per bin, one channel each, in the order given. See <code>grain_matrix()</code> for the seven and for what separates an extreme reading from an extreme day.
<code>year_start</code>	"MM-DD" boundary of the hydrological year, used by "season" and "year".
<code>lag</code>	The gap between a target's instant and the end of its lookback.
<code>bins</code>	Sub-bins the lookback is cut into, oldest first. One gives a block of features, several give a sequence.
<code>x</code>	A named list of representations, a single representation, or a character vector of grain names.

Details

`grains("auto")` is the set of named grains the record gives at least two bins, in the order native, halfday, day, week, month, season, year, leaving out any grain the requested statistics are not defined at. There is no cap on it: over three years of hourly readings native is 26304 bins, and a caller who does not want that names the grains instead.

Value

A `timesift_sift`: a named list of representations, labelled by each one's own label where the list was not named.

Examples

```
grains("day", "week", "month")
grains(c("month", "year"), stats = c("cold_day", "mean", "warm_day"))
lookbacks("30 days", "90 days", bins = 3L)
```

grain_contrasts

Compare every grain against a learner's best one

Description

A mixed model on the per-cell scores of one learner, $\text{score} \sim \text{grain} + (1 \mid \text{variable}) + (1 \mid \text{fold})$, fitted by restricted maximum likelihood. Every grain is then compared against the reference by Dunnett's many-to-one procedure, which corrects for the comparisons made without correcting for pairs nobody asked about.

Usage

```
grain_contrasts(ladder, learner = NULL, reference = NULL, adjust = "mvt")
```

Arguments

ladder	A <code>grain_ladder()</code> result.
learner	Which learner's grid to fit. The only one in the ladder by default.
reference	The grain every other is compared against. The learner's best by default.
adjust	Multiplicity adjustment passed to <code>emmeans::contrast()</code> .

Details

The design is balanced across variables, folds and grains, so each grain is compared within a variable and within a fold and the variation between variables cancels from the comparison. That is what lets a difference of 0.015 hold up where absolute skill ranges across variables by ten times as much.

Taking the best-observed grain as the reference is a choice that favours the reference, so read this beside the paired differences `paired_contrast()` gives, which single out no grain.

Value

A data frame of one row per grain: the estimated marginal mean difference from the reference, its interval, and the adjusted p-value. Needs `lme4`, `lmerTest` and `emmeans`.

Examples

```
set.seed(1)
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 200)
units <- sprintf("p%03d", 1:60)
warmth <- rnorm(60)
d <- data.frame(
  plot = rep(units, each = length(t)), t = rep(t, length(units)),
  temp = as.numeric(vapply(warmth, function(w) 1.5 * w + rnorm(length(t), sd = 20),
    numeric(length(t))))
)
y <- matrix(rbinom(60 * 6, 1, plogis(3 * as.numeric(outer(warmth, rep(c(1, -1), 3))))),
  ncol = 6, dimnames = list(units, paste0("sp", 1:6)))
x <- grain_matrix(d, plot, t, temp, grain = c("day", "week", "month"))
lad <- grain_ladder(x, y, elasticnet(), folds = fold_map(y, v = 5), verbose = FALSE)
grain_contrasts(lad)
```

grain_ladder

Fit at every grain and see where skill saturates

Description

Cross-validates every learner at every grain of a representation set, on one fold map and one mask of scorable cells, and returns the score of each (grain, learner, variable, fold) cell. It is the measurement the package exists for: how much of a record a model needs, read off the point where making the record finer stops paying.

Usage

```
grain_ladder(
  x,
  y,
  learners,
  folds = NULL,
  response = "presence_absence",
  metric = NULL,
  control = train_control(),
  keep_fits = FALSE,
  interval = c("variables", "nested_cv"),
  repeats = 1L,
  seed = 1L,
  verbose = TRUE
)

## S3 method for class 'timesift_ladder'
summary(object, ...)
```

Arguments

x	A grain_matrix() result, a timesift_set() , or a named list of representations.
y	The response for the same units.
learners	A learner, a set of them from c() , a list, or names of registered ones. An unnamed set is labelled by each learner's own name.
folds	A fold map from fold_map() , or any named integer vector. Built with the defaults of fold_map() when not given.
response	Name of the registered response head.
metric	Name of a registered metric, or a function of (y, p), or NULL for the response head's own. Whichever it is, it travels with the fit and is what every later rescoring reads; a function is reported as <code><function></code> .
control	train_control() , the training settings every neural learner of the ladder reads. A learner carrying a control of its own overrides it on the settings that control names.
keep_fits	Keep every per-fold fitted model, which is what lets occlusion() read a fitted model without refitting it.
interval	"nested_cv" refits every arm inside every outer training set, which is what paired_contrast() reads an interval for the difference in risk off; see select_grain() for what it estimates and what it costs. "variables", the default, fits nothing further, and a contrast is then read across the response variables.
repeats	Repetitions of the nested cross-validation, each on its own fold map. The first is the map the ladder was cross-validated on.
seed	Seed the repetitions' fold maps are drawn under. Two tables whose contrast is to be read take the same folds, repeats and seed.

verbose	Report each arm and each fold as it runs.
object	A ladder.
...	Ignored, so that <code>summary()</code> takes the arguments its generic declares.

Details

Every arm sees identical splits and is restricted to identical cells, so the arms' means share a denominator and any two of them can be compared cell by cell with `paired_contrast()`.

Value

A data frame of one row per scored cell, of class `timesift_ladder`, carrying the grain, the learner, the variable, the fold and the score. The held-out prediction of every unit is kept in the `predictions` attribute, and the scorable-cell mask in `cells`.

Examples

```
set.seed(1)
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 200)
units <- sprintf("p%02d", 1:60)
warmth <- rnorm(60)
d <- data.frame(
  plot = rep(units, each = length(t)), t = rep(t, length(units)),
  temp = as.numeric(vapply(warmth, function(w) w + sin(seq_along(t) / 300) + rnorm(length(t))),
    numeric(length(t))))
y <- matrix(rbinom(120, 1, plogis(c(warmth, -warmth))), nrow = 60,
  dimnames = list(units, c("sp1", "sp2")))
x <- grain_matrix(d, plot, t, temp, grain = c("week", "month"))
lad <- grain_ladder(x, y, elasticnet(), folds = fold_map(y, v = 3), verbose = FALSE)
summary(lad)
```

grain_matrix

Reduce sensor series to a temporal grain

Description

Bins each unit's readings by the calendar and summarises every bin by one or more statistics, returning the array a model is fitted on. The reduction is the choice the package exists to make explicit: `grain` sets how coarse the record becomes, `stats` sets what survives the reduction, and the two are not interchangeable.

Usage

```
grain_matrix(
  data,
  id,
  time,
```

```

    value,
    grain = "day",
    stats = "mean",
    year_start = "09-01",
    partial = c("keep", "drop")
  )

```

Arguments

<code>data</code>	A data frame of readings in long form, one row per reading.
<code>id</code>	Column identifying the unit carrying the sensor. A bare column name or a string.
<code>time</code>	Column of reading instants, POSIXct. A bare column name or a string.
<code>value</code>	Column of readings, numeric. A bare column name or a string.
<code>grain</code>	One of "native", "halfday", "day", "week", "month", "season", "year". The four coarse grains follow the calendar, so a bin is a real week or month rather than a fixed block of hours. Naming several grains returns one representation per grain, a <code>timesift_set()</code> . A function is called on the reading instants and must return the POSIXct start of each reading's bin, which is how a calendar the package does not carry, such as astronomical seasons, is binned.
<code>stats</code>	Statistics to compute per bin, one channel each, in the order given. See Details.
<code>year_start</code>	"MM-DD" boundary of the hydrological year, used by "season" and "year". Defaults to "09-01".
<code>partial</code>	What to do with a bin the record does not cover for its whole calendar span, which is what a record beginning or ending away from a bin boundary produces. "keep", the default, returns it alongside the full bins; "drop" removes it. See Partial bins.

Details

Seven statistics are available, and the distinction between an extreme reading, an extreme day and a typical day is deliberate rather than pedantic:

- `mean`: arithmetic mean of the readings in the bin.
- `min`, `max`: coldest and warmest single reading in the bin.
- `cold_day`, `warm_day`: coldest and warmest day, each day first reduced to its own mean. Defined for "day" and coarser.
- `mean_daily_min`, `mean_daily_max`: the bin's average daily minimum and average daily maximum, each day first reduced to its own extreme. Defined for "day" and coarser.

An extreme day is a state the unit was in; an extreme reading can be one hour; an average daily extreme is the exposure a typical day of the bin brought. On alpine soil temperature the day-level pair carries more predictive signal than the bin mean, and by more the coarser the bin.

Whether a grain is a day or coarser is decided from the bins rather than from the grain's name, so a supplied calendar that cuts inside a day is refused for these four as well, naming the day it splits.

Nothing is standardised here. Scaling belongs to the fold it is computed on, never to the representation, because computing it over all units would leak held-out units into the input.

Value

A numeric array of shape [unit, bin, channel], with dimnames giving the sorted unit identifiers, the ISO-8601 start of each bin, and the statistic names. Attributes:

- grain: the grain name.
- stats: the statistic names in channel order.
- year_start: the boundary used.
- bin_start: the instant each bin begins on the calendar, which is earlier than the bin's first reading wherever the record does not reach the boundary.
- bin_end: the last reading instant assigned to each bin.
- bin_n: a [unit, bin] matrix of how many readings fell in each bin.
- bin_partial: a logical vector marking the bins the record does not cover for their whole calendar span.

Naming more than one grain returns a `timesift_set()` of those arrays.

Time zone

Bins follow the calendar the series is carried in, which is the `tzzone` attribute of `time`; a column with `none` is read as UTC, and a name the zone database does not know is an error. The zone is resolved once, at the edge: below it the binning works in local time, where a day is 86400 seconds whatever the night did, so a zone that moves its clock at midnight has no midnight to lose. A bin start is a local time, so reporting it back as an instant needs a rule: one the clock skipped resolves to the instant the clock jumped to, one the clock repeated to the first of the two. Instants are read at whole seconds.

In a zone that keeps daylight saving time a day is therefore a wall-clock day: the day the clock goes forward spans 23 hours and the day it goes back 25, and the week and the month holding them one hour less or more. Hourly readings carried in "Europe/Vienna" give a 2021-03-28 day of 23 readings and a 2021-10-31 day of 25. A record kept on a fixed offset, as the Schrankogel deposit is (26,304 hourly readings, 1,096 days of 24), bins into days of 24 hours; a series carried in a daylight-saving zone bins that way too once its `tzzone` is set to the fixed offset of standard time, such as "Etc/GMT-1" for Central European Time, whose days then run from 01:00 to 01:00 on the summer clock.

The "native" grain is the one grain not read on that clock: its bin is the reading itself, so the two readings of an hour a zone repeats are two bins, and the record read at "native" is the same array whichever zone it is carried in.

Bins that do not tile the record

Every unit must reach every bin, and consecutive bins must be one bin apart on the grain's own calendar. A bin no unit reaches is never built, so a month missing from the whole record would otherwise pass as four adjacent monthly bins with one simply gone. Neither the "native" grain, whose bin is the reading itself, nor a supplied calendar, which declares its own bin lengths, is held to the second rule. `coverage()` lays the same binning out as a count of readings per unit and bin, which is where a refused record's gaps are read off.

Partial bins

A bin is partial when the record does not cover its whole calendar span. Which bins those are follows from where the record starts and stops against the calendar, not from the grain alone: three years of hourly readings from 1 September carry no partial month and no partial season on a "09-01" boundary, but the same record carries a partial week at each end, because 1 September is a Wednesday. A record from an arbitrary deployment date carries one at each end of almost every grain.

A bin is partial if its start precedes the first reading of the record, or if its calendar span runs past the last reading plus the record's own sampling interval, taken as the smallest gap between consecutive distinct reading instants. Only a bin at an end of the record can satisfy either, because every unit is required to span every bin in between. The verdict is returned as the `bin_partial` attribute whichever way partial is set, so a kept partial bin is labelled rather than silent.

Keeping partial bins is the default because dropping them discards the record's ends: on a seasonal grain that is up to three months of readings at each end. The cost of keeping them is that such a bin's mean is taken over fewer readings and its extremes over fewer days, so `cold_day` and `warm_day` there are drawn from a shorter draw and sit closer to the bin mean than a full bin's would. `bin_n` gives the count the bin was actually reduced from.

A caller-supplied binning declares its own bins, so the package cannot know where the last one was meant to end and takes the record's end as its end. Such a final bin is never reported partial; its leading bin is judged as any other.

Examples

```
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 40)
d <- data.frame(plot = rep(c("a", "b"), each = length(t)),
               t = rep(t, 2),
               temp = c(sin(seq_along(t) / 24), cos(seq_along(t) / 24)))
x <- grain_matrix(d, plot, t, temp, grain = "week",
               stats = c("cold_day", "mean", "warm_day"))

dim(x)
dimnames(x)[[3]]

ladder <- grain_matrix(d, plot, t, temp, grain = c("day", "week"))
names(ladder)
```

implied_skill

What population skill a reported level is consistent with

Description

`tss_inflation()` maps a population skill to the level a design reports for it. This inverts that map: given a level actually read off a ladder, it solves for the population skill whose expected reported level equals it.

Usage

```
implied_skill(
  y,
  folds,
  observed,
  grid = seq(0, 0.95, by = 0.05),
  replicates = 200L,
  seed = 1L
)
```

Arguments

y	The response: a matrix or data frame of units by variables, with unit identifiers in the row names or in a leading character or factor column.
folds	A fold map from <code>fold_map()</code> , or any named integer vector of the same shape.
observed	Reported levels to invert.
grid	Population skills the forward map is measured on before interpolating between them.
replicates	Replicates per value.
seed	Random seed.

Details

It answers the question a level raises once the inflation is known, under the distribution of predictions `tss_inflation()` plants; a model whose predictions are distributed otherwise is inflated by a different amount. It does not correct a difference between two arms, whose inflations need not be equal.

Value

A data frame of one row per observed level: the level, the population skill it is consistent with, and whether that sits inside the grid the map was measured on.

Examples

```
set.seed(1)
y <- matrix(rbinom(1200, 1, 0.15), nrow = 200,
            dimnames = list(sprintf("p%03d", 1:200), paste0("sp", 1:6)))
implied_skill(y, fold_map(y, v = 5), observed = 0.71, replicates = 40)
```

kappa_score	<i>Cohen's kappa, and where two models disagree</i>
-------------	---

Description

A chance-corrected agreement rate on a two-by-two table. Read against the observed response it is a skill score beside `tss()`; read between two models' decisions on the same units it says where the two part company.

Usage

```
kappa_score(y, p, rule = c("youden", "kappa", "prevalence"))

decision_threshold(y, p, rule = c("youden", "kappa", "prevalence"))

model_agreement(y, p_a, p_b, rule = c("youden", "kappa", "prevalence"))
```

Arguments

y	Observed presence-absence, 0/1 or logical.
p	Predicted scores for the same units, in the same order. Higher means presence.
rule	Threshold rule: "youden", "kappa" or "prevalence".
p_a, p_b	Two models' predictions for the same units.

Details

Kappa is read at a threshold rather than maximised over one, so the rule that picks the threshold is part of the statistic. "youden" is the operating point `tss()` is defined at and inherits its selection bias; "kappa" maximises kappa itself and inherits the analogous bias; "prevalence" cuts at the observed presence rate, which selects nothing from the labels and is the rule to read an absolute level at.

Value

For `kappa_score()`, one number. For `decision_threshold()`, the cut itself, applied as $p \geq \text{threshold}$. For `model_agreement()`, a one-row data frame carrying the agreement kappa between two models cut by the same rule, the share of units they decide differently, and how often each is the one that is right there.

Examples

```
y <- c(0, 0, 0, 1, 1, 1)
kappa_score(y, c(0.1, 0.2, 0.6, 0.4, 0.8, 0.9))
model_agreement(y, c(0.1, 0.2, 0.6, 0.4, 0.8, 0.9), c(0.2, 0.1, 0.3, 0.7, 0.9, 0.8))
```

 learner

Define a learner

Description

A learner is a pair of functions: one that fits a model to a representation and a response, and one that predicts from it. Everything the package fits goes through this pair, so a learner of your own sits beside the ones that ship and needs no change to the ladder, the folds or the scoring.

Usage

```
learner(
  name,
  fit,
  predict,
  data = NULL,
  reads = c("tabular", "sequence"),
  multi = c("separate", "joint"),
  control = NULL,
  needs = character(),
  params = list()
)
```

Arguments

name	Name the learner is reported under.
fit	A function of (x, y, ...), where x is a [unit, bin, channel] array and y the response matrix for the same units, returning a fitted object. A fit that declares a control argument is handed the resolved <code>train_control()</code> , and one that declares a head argument is handed the registered response head, whose loss and activation say what it is fitting toward. The learners that ship read both from there and hold no response of their own. One that declares a group argument is handed the grouping the outer fold map keeps whole, one value per unit of x or NULL, so a split it draws inside the fit keeps the same groups whole.
predict	A function of (model, x) returning a [unit, variable] matrix of predictions for the units of x, in that order.
data	A representation the learner is pinned to, or NULL to run across every representation of the run.
reads	"tabular" or "sequence".
multi	"separate" where the learner fits one model per response, "joint" where one model covers them all. It is handed the whole response matrix either way; this is what the learner says it does with it, and what a report says of the candidate.
control	A <code>train_control()</code> for this learner alone. The settings it names override the control the run was given; everything else is taken from that one.

needs	Packages the learner requires. A learner that cannot run says so at once rather than falling back to something else.
params	Settings carried with the learner and passed to fit.

Details

A learner declares what it can be handed and how it covers several responses. `reads` is "tabular" where the bins reach it as a block of predictors and "sequence" where their order in time is what it reads, and `multi` is "joint" where one fitted model covers every response and "separate" where the learner fits one per response. Either way it is handed the whole response matrix and returns one column per response, so nothing above the learner layer has to know which it was, and the block of predictors is built once for the fit rather than once for every response of it.

`data` pins a learner to one representation. Left NULL the learner runs across every representation of the run.

Value

A `timesift_learner`.

Examples

```
# The bin means of a unit, fed to one logistic regression per variable.
flat_glm <- learner(
  "flat_glm",
  fit = function(x, y, ...) {
    f <- as.data.frame(apply(x, c(1, 3), mean))
    lapply(seq_len(ncol(y)), function(j)
      stats::glm(y[, j] ~ ., data = f, family = stats::binomial()))
  },
  predict = function(model, x) {
    f <- as.data.frame(apply(x, c(1, 3), mean))
    vapply(model, function(m) stats::predict(m, f, type = "response"), numeric(nrow(f)))
  }
)
flat_glm
```

lookback_matrix

Reduce sensor series to a lookback anchored on each target

Description

Reads, for every target, a fixed length of record ending a fixed lag before that target's own instant, and summarises it by one or more statistics. It is the reduction a calendar cannot express: two targets on the same unit a fortnight apart read two different stretches of the same series, so the bins are relative to the target rather than to a month or a week.

Usage

```
lookback_matrix(
  data,
  id,
  time,
  value,
  at,
  span,
  lag = "0 days",
  bins = 1L,
  stats = "mean"
)
```

Arguments

<code>data</code>	A data frame of readings in long form, one row per reading.
<code>id</code>	Column identifying the unit carrying the sensor. A bare column name or a string.
<code>time</code>	Column of reading instants, POSIXct. A bare column name or a string.
<code>value</code>	Column of readings, numeric. A bare column name or a string.
<code>at</code>	A data frame of targets with an <code>id</code> column naming the unit and an <code>at</code> column of anchor instants, POSIXct. One row per target; a unit may carry any number of them. The columns are read by name, as every alignment in the package is.
<code>span</code>	The lookback's length, as a duration. See Durations.
<code>lag</code>	The gap between the anchor and the end of the lookback, as a duration. Defaults to "0 days", which ends the lookback at the anchor itself.
<code>bins</code>	How many sub-bins the lookback is cut into, oldest first. <code>span</code> must divide by it exactly. One bin gives a block of features; several give a sequence a convolution can read.
<code>stats</code>	Statistics to compute per bin, one channel each, in the order given. The same seven <code>grain_matrix()</code> carries.

Details

Bin b of a target anchored at a covers $[a - \text{lag} - \text{span} + b * \text{step}, a - \text{lag} - \text{span} + (b + 1) * \text{step})$, with `step` the lookback's length divided by `bins` and b counted from zero. The interval is closed at the left and open at the right, so a reading on a boundary belongs to the later bin, and only the readings of the target's own unit are read.

Every `(target, bin)` cell must hold at least one reading. A lookback reaching past either end of the record is an error naming the target and the interval, never a padded row: an invented value in front of a model is worse than a target the record cannot answer for.

The four day-level statistics reduce each calendar day first, so they are defined only where every day lies whole inside one bin. For a lookback that is two conditions rather than one: `step` must be a whole number of days, and $a - \text{lag} - \text{span}$ must fall on a day boundary. Either failing is an error naming the target.

Value

A numeric array of shape `[target, bin, channel]`, of class `timesift_matrix`. Its rows are the rows of `at`, in `at`'s own order, named by `at`'s row names. Its bins are named by where each one opens relative to the anchor, oldest first, and its channels by the statistic. Attributes:

- `grain`: "lookback".
- `span`, `lag`: the durations, resolved to seconds.
- `bins`: how many bins the lookback was cut into.
- `stats`: the statistic names in channel order.
- `bin_n`: a `[target, bin]` matrix of how many readings fell in each cell.

Durations

`span` and `lag` are read from a count and a unit – "30 days", "12 hours", "1 year" – or from a bare number of seconds. A **year is 365 days and a month is 30 days** here. A lookback of a fixed length is a fixed length, not a calendar step: the point of anchoring on the target is that every target reads the same amount of record, which a February and a leap year would take away. Where the calendar is what matters, `grain_matrix()` is the call that follows it.

The units are seconds, minutes, hours, days, weeks, months and years, singular or plural.

Time zone

The calendar is the series', taken from the `tz` attribute of time as `grain_matrix()` takes it; a column with `none` is read as UTC. The anchors are instants and are read as a clock in that same calendar, whatever zone `at` carries, so one record is binned by one calendar. The span is measured on that clock: a lookback of one day ending at a local midnight holds the whole local day before it, which is 25 hours of record on the night a zone sets its clock back and 23 on the night it sets it forward. That is what keeps a calendar day whole inside a bin for the four day-level statistics; a length fixed in instants could not.

See Also

`grain_matrix()`, the reduction that follows the calendar instead.

Examples

```
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 60)
d <- data.frame(plot = rep(c("a", "b"), each = length(t)),
               t = rep(t, 2),
               temp = c(sin(seq_along(t) / 24), cos(seq_along(t) / 24)))
at <- data.frame(id = c("a", "b"),
               at = as.POSIXct(c("2021-10-20", "2021-10-25"), tz = "UTC"))
x <- lookback_matrix(d, plot, t, temp, at = at, span = "30 days", bins = 3L,
                  stats = c("cold_day", "mean", "warm_day"))

dim(x)
dimnames(x)[[2]]
```

 native

How a series becomes an array a learner reads

Description

A representation is the reduction the package exists to make explicit: the record unreduced, the record at a calendar grain, several grains bound into one block of features, or a lookback of fixed length ending at each target's own instant. It carries the settings and nothing else, so the same object describes a representation before any record has been seen, names the arm it produced in a fitted object, and rebuilds itself for new targets in `predict.timesift()`.

Usage

```
native(stats = "mean", year_start = "09-01")

grain(grain, stats = "mean", year_start = "09-01")

multigrain(grains = NULL, stats = "mean", year_start = "09-01")

lookback(span, lag = "0 days", bins = 1L, stats = "mean")
```

Arguments

stats	Statistics computed per bin, one channel each, in the order given. See <code>grain_matrix()</code> for the seven and for what separates an extreme reading from an extreme day.
year_start	"MM-DD" boundary of the hydrological year, used by "season" and "year".
grain	One of "native", "halfday", "day", "week", "month", "season", "year", or a function of the reading instants returning each reading's bin start. See <code>grain_matrix()</code> .
grains	Grains bound side by side into one block, or NULL for the automatic set <code>grains()</code> describes.
span	The lookback's length, as a duration such as "30 days" or a number of seconds. See <code>lookback_matrix()</code> for how a duration is read.
lag	The gap between a target's instant and the end of its lookback.
bins	Sub-bins the lookback is cut into, oldest first. One gives a block of features, several give a sequence.

Details

Every representation carries `label`, the name it is reported under; `kind`, one of "grain", "multigrain" and "lookback"; the settings its kind uses; and `sequence`, which says whether its bins are ordered in time and so mean something to a convolution. `native()`, `grain()` and a `lookback()` of more than one bin are sequences; `multigrain()` and a one-bin `lookback()` are blocks of features.

`multigrain()` flattens each of its grains to one row per target and puts them side by side, so a column of the block names the grain, the statistic and the bin it came from. It is the tabular representation a penalised regression or a random forest reads.

Value

A timesift_representation.

Examples

```
native()
grain("week", stats = c("cold_day", "mean", "warm_day"))
multigrain(c("month", "season"))
lookback("30 days", bins = 3L)
```

occlusion	<i>What part of the record a fitted model reads</i>
-----------	---

Description

Holds one bin of the record back at a time, rescores the held-out units, and records the fall in score as that bin's weight. Nothing is refitted: the models kept by `grain_ladder(keep_fits = TRUE)` are the ones read, so the profile describes the models that produced the reported scores rather than a fresh set of them.

Usage

```
occlusion(x, ...)

## Default S3 method:
occlusion(x, ...)

## S3 method for class 'timesift_ladder'
occlusion(
  x,
  data,
  y,
  arm,
  over = c("bin", "channel"),
  substitute = c("permute", "fold_mean", "unit_mean"),
  metric = NULL,
  permutations = 20L,
  seed = 1L,
  ...
)

## S3 method for class 'timesift'
occlusion(x, candidate, over = c("bin", "channel"), ...)
```

Arguments

x	A <code>timesift()</code> fit, or a <code>grain_ladder()</code> result, in either case fitted with <code>keep_fits = TRUE</code> .
...	Passed to the method.
data	The representation set the ladder was fitted on.
y	The response it was fitted to.
arm	The arm to read, as "grain learner" or "learner".
over	"bin" to hold each bin back in turn, "channel" for each channel.
substitute	What a held-back part is replaced by: "permute", "fold_mean" or "unit_mean".
metric	Name of the registered metric the rescoring is read by, or a function of (y, p). Left unset it is the one the fit was scored under, so a weight is a fall in the number <code>summary()</code> reports rather than in a second one. "roc_auc" is usually the steadier reading over many rescorings: it responds to every reordering of the units, where a maximum over thresholds frequently does not move at all.
permutations	Draws averaged over, for substitute = "permute".
seed	Random seed.
candidate	Name of the candidate to read, for a run.

Details

A model has to be shown something in place of a held-back bin, and what it is shown decides what the weight means. Permuting the bin's values across units keeps the observed readings exactly and cuts only the link between a reading and its unit. Replacing every unit by the fitting-fold mean removes all between-unit variation while keeping the shape of the year. Replacing the bin by each unit's own mean over the record keeps how warm a unit is and removes only that bin's departure from it.

A bin is held back whole: a permutation moves every channel of the bin together, so a unit is shown another unit's week rather than a coldest day from one unit beside a warmest day from another. A channel that is the same for every unit, as `calendar_channels()` are, says where the bin sits rather than what a unit read there, and is left in place.

Read with `over = "channel"` the same machinery asks what each statistic of a grain carries, holding one channel back across the whole record instead of one bin across all channels.

Reached through a `timesift()` run rather than through a ladder, the profile reads the per-fold models the run was told to keep, so every bin is held back from a model that never saw the units it is rescored on. The candidate is named as `summary()` reports it.

Value

A data frame of one row per held-back part and variable, carrying the mean weight over folds and the score with and without the part.

Examples

```
set.seed(1)
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 120)
units <- sprintf("p%02d", 1:40)
warmth <- rnorm(40)
d <- data.frame(
  plot = rep(units, each = length(t)), t = rep(t, length(units)),
  temp = as.numeric(vapply(warmth, function(w) w + sin(seq_along(t) / 300) + rnorm(length(t)),
    numeric(length(t))))))
y <- matrix(rbinom(80, 1, plogis(c(warmth, -warmth))), nrow = 40,
  dimnames = list(units, c("sp1", "sp2")))
x <- grain_matrix(d, plot, t, temp, grain = "month")
lad <- grain_ladder(x, y, elasticnet(), folds = fold_map(y, v = 3),
  keep_fits = TRUE, verbose = FALSE)
head(occlusion(lad, x, y, "month|elasticnet", permutations = 3))
```

paired_contrast	<i>Compare two arms cell by cell</i>
-----------------	--------------------------------------

Description

Two arms scored on the same held-out units do not necessarily have the same set of defined cells, so a difference of two marginal means is not a difference between the arms. This takes the difference inside each (variable, fold) cell both arms scored, averages it within a variable over its folds, and summarises those per-variable means, the variables being the independent replicates.

Usage

```
paired_contrast(ladder, a, b, interval = c("variables", "nested_cv"))
```

Arguments

ladder	A <code>grain_ladder()</code> result.
a, b	The two arms, each named "grain learner".
interval	Which interval the row carries. "variables" is the spread across the response variables of this dataset. "nested_cv" is an interval for the difference in the two arms' risk on a new sample, by the nested cross-validation of Bates, Hastie and Tibshirani (2024) read on the difference of the two arms' cell scores, which needs a ladder fitted with <code>grain_ladder(interval = "nested_cv")</code> . The paper gives the estimator for one procedure's error and names the difference between two as an extension; the identity its estimator rests on is on the difference of two losses as much as on one.

Details

Pairing removes the variation between variables and the part of a threshold-selected metric's bias that the design sets, but not the part that belongs to each arm. TSS read at the threshold that maximises it is biased upward where presences are thin, and how far depends on how an arm's predictions are distributed as well as on how many presences the cell holds, so two arms of equal skill on the same cell can carry different biases. A difference in TSS can therefore favour one arm with no difference in skill behind it. A threshold-free metric such as `roc_auc()` has no cut to choose, and a TSS contrast is best read beside the same contrast under it.

An arm is named whole, by its grain and its learner. A learner named alone would have to take its best grain, and that grain is chosen on the held-out scores the contrast is then read off: the difference becomes one between two maxima, favouring whichever learner ran across more grains, and neither the interval nor the p-value accounts for the choice. It is the mechanism `tss_inflation()` measures one level down, and here pairing does not cancel it. `select_grain()` chooses a grain on inner folds instead, and its `compare` argument contrasts the selection with the arms of a ladder on matched cells.

Value

A one-row data frame: the mean per-variable difference; the centre of the interval and its bounds, on Student's t across variables or on nested cross-validation as interval names, which the row carries; the number of variables the difference favours; the paired cells and variables it rests on; a Wilcoxon signed-rank p-value; and `p_method`, "exact" where the p-value is read off the exact distribution and "normal" where it is the normal approximation with continuity and tie corrections, which it is when the per-variable differences hold a zero or a tie or number fifty or more.

References

Bates, S., Hastie, T. and Tibshirani, R. (2024). Cross-validation: what does it estimate and how well does it do it? *Journal of the American Statistical Association* **119**(546), 1434-1445. doi:10.1080/01621459.2023.2197686

Examples

```
set.seed(1)
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 200)
units <- sprintf("p%02d", 1:60)
warmth <- rnorm(60)
d <- data.frame(
  plot = rep(units, each = length(t)), t = rep(t, length(units)),
  temp = as.numeric(vapply(warmth, function(w) w + sin(seq_along(t) / 300) + rnorm(length(t)),
    numeric(length(t))))
y <- matrix(rbinom(120, 1, plogis(c(warmth, -warmth))), nrow = 60,
  dimnames = list(units, c("sp1", "sp2")))
x <- grain_matrix(d, plot, t, temp, grain = c("week", "month"))
lad <- grain_ladder(x, y, elasticnet(), folds = fold_map(y, v = 3), verbose = FALSE)
paired_contrast(lad, "week|elasticnet", "month|elasticnet")
```

plot.timesift	<i>Draw a run</i>
---------------	-------------------

Description

One line per learner across the representations it ran on, read the way a ladder is read, and the stack's held-out score drawn across them, its weights fitted inside each outer training fold. The curves are scored on the folds a choice among them would be judged on, so the best of them sits a little high; the ensemble line does not. Where the ensemble line sits above every curve the candidates are carrying different parts of the signal, and where it sits on or below the best curve they are not.

Usage

```
## S3 method for class 'timesift'
plot(x, col = NULL, interval = TRUE, ...)
```

Arguments

x	A timesift result.
col	One colour per learner, recycled.
interval	Draw the interval across responses.
...	Passed to <code>graphics::plot()</code> .

Value

The table the plot is drawn from, invisibly.

plot.timesift_ladder	<i>Draw a ladder</i>
----------------------	----------------------

Description

One line per learner across the grains, at the across-variable mean of the per-variable score, with a 95 percent interval from its standard error across variables, on Student's t with one degree of freedom fewer than there are variables. An open circle marks each learner's best grain, which is where the curve says the record stops paying for being read more finely.

Usage

```
## S3 method for class 'timesift_ladder'
plot(x, col = NULL, interval = TRUE, ...)
```

Arguments

x	A <code>grain_ladder()</code> result.
col	One colour per learner, recycled.
interval	Draw the interval across variables.
...	Passed to <code>graphics::plot()</code> .

Value

The summary table the plot is drawn from, invisibly.

Examples

```
set.seed(1)
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 200)
units <- sprintf("p%02d", 1:60)
warmth <- rnorm(60)
d <- data.frame(
  plot = rep(units, each = length(t)), t = rep(t, length(units)),
  temp = as.numeric(vapply(warmth, function(w) w + sin(seq_along(t) / 300) + rnorm(length(t))),
    numeric(length(t))))
y <- matrix(rbinom(120, 1, plogis(c(warmth, -warmth))), nrow = 60,
  dimnames = list(units, c("sp1", "sp2")))
x <- grain_matrix(d, plot, t, temp, grain = c("day", "week", "month"))
lad <- grain_ladder(x, y, elasticnet(), folds = fold_map(y, v = 3), verbose = FALSE)
plot(lad)
```

```
plot.timesift_selection
```

Draw how stable the choice of grain was

Description

Every candidate's inner score in every outer fold, one line per outer fold, with an open circle on the candidate that fold selected. A selection that lands on the same candidate each time draws its circles in one column; one that wanders says the grid is flat enough that the choice is arbitrary, which is worth seeing beside the estimate rather than after it.

Usage

```
## S3 method for class 'timesift_selection'
plot(x, col = NULL, ...)
```

Arguments

x	A <code>select_grain()</code> result.
col	One colour per outer fold, recycled.
...	Passed to <code>graphics::plot()</code> .

Value

The table of inner scores the plot is drawn from, invisibly, with `at`, the position of each candidate on the axis, and `selected`, whether its fold chose it.

Examples

```
set.seed(1)
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 200)
units <- sprintf("p%02d", 1:60)
warmth <- rnorm(60)
d <- data.frame(
  plot = rep(units, each = length(t)), t = rep(t, length(units)),
  temp = as.numeric(vapply(warmth, function(w) w + sin(seq_along(t) / 300) + rnorm(length(t)),
    numeric(length(t))))
y <- matrix(rbinom(120, 1, plogis(c(warmth, -warmth))), nrow = 60,
  dimnames = list(units, c("sp1", "sp2")))
x <- grain_matrix(d, plot, t, temp, grain = c("week", "month"))
sel <- select_grain(x, y, elasticnet(), folds = fold_map(y, v = 3), inner = 3,
  verbose = FALSE)
plot(sel)
```

positive_weights	<i>Case weights that balance a rare response</i>
------------------	--

Description

The weight every learner that ships fits a presence-absence response under: each presence of a response weighs the ratio of absences to presences among the fitting units, capped, and each absence weighs one. A response with a presence in one target of a hundred is otherwise fitted away by any learner that minimises a mean loss, and the encoders, the penalised fit, the forest and the forward search would each have to decide that for themselves.

Usage

```
positive_weights(y, cap = 50, fitting = NULL)
```

Arguments

<code>y</code>	The response matrix, <code>[unit, variable]</code> , 0/1.
<code>cap</code>	Ceiling on the weight a presence is given, at least one.
<code>fitting</code>	A logical vector over the rows of <code>y</code> , TRUE for the units the model is fitted on, or NULL for all of them. The ratio is read off those rows alone.

Details

The ratio is read off the units the model is fitted on and the weight applies to every unit handed in. An encoder holds part of its units back as an inner validation set and reads the loss it stops on under the same weights, so the loss that stops the fit is the loss the fit minimises; a unit held back that way is not in the count the ratio is made from.

The weights are the response head's: the shipped presence-absence head carries this function as its weights, and a head registered with `weights = function(y, fitting) positive_weights(y, cap = 20, fitting = fitting)` weights every learner by that cap instead. A head without weights is fitted unweighted.

Value

A numeric `[unit, variable]` matrix of case weights, one per cell of `y`.

Examples

```
y <- cbind(rare = c(1, 0, 0, 0, 0, 0), common = c(1, 1, 1, 0, 0, 0))
positive_weights(y)
positive_weights(y, fitting = c(TRUE, TRUE, TRUE, TRUE, FALSE, FALSE))
```

<code>predict.timesift</code>	<i>Predict from a fitted timesift</i>
-------------------------------	---------------------------------------

Description

Rebuilds every member's representation for the new targets from the settings its own arm was built with, predicts with the model refitted on all targets, and combines them where the ensemble is asked for.

Usage

```
## S3 method for class 'timesift'
predict(object, targets, series = NULL, candidate = "ensemble", ...)
```

Arguments

<code>object</code>	A <code>timesift()</code> fit.
<code>targets</code>	A data frame of targets, carrying the identifier, the anchor and the static columns the fit was given.
<code>series</code>	The long table of readings for those targets, or <code>NULL</code> for a targets-only fit.
<code>candidate</code>	"ensemble", the stack refitted on every target; "selected", the candidate the rule chose on every target (<code>object\$choice</code>); or the name of one candidate.
<code>...</code>	Ignored.

Value

A [target, response] matrix of predictions, named by target and in the order the fit carries its own targets: sorted by identifier, or the targets' own order where target_time anchors them.

register_learner	<i>Register a learner</i>
------------------	---------------------------

Description

Makes a learner available by name to [grain_ladder\(\)](#) and to [learners\(\)](#). The learners that ship are registered the same way, so there is no list of names inside the fitting code.

Usage

```
register_learner(name, constructor, overwrite = FALSE)

learners()
```

Arguments

name	Name the learner is asked for by.
constructor	A function returning a learner() .
overwrite	Replace an existing registration.

Value

The constructor, invisibly.

Examples

```
learners()
```

register_metric	<i>Register a metric</i>
-----------------	--------------------------

Description

A metric scores one held-out cell: the observed response of the units in a fold and a model's predictions for them. Registering one makes it available to [grain_ladder\(\)](#) by name, with no change to the fitting code.

Usage

```
register_metric(name, fn, overwrite = FALSE)

metrics()
```

Arguments

name	Name the metric is asked for by.
fn	A function of (y, p), the observed values and the predictions for one cell, both vectors of the same length, returning one number or NA where the cell defines none.
overwrite	Replace an existing registration.

Value

The registered function, invisibly.

Examples

```
register_metric("hit_rate", function(y, p) mean((p >= 0.5) == (y == 1)), overwrite = TRUE)
"hit_rate" %in% metrics()
```

register_response	<i>Register a response head</i>
-------------------	---------------------------------

Description

A response head says what the values being predicted are, how they reach a learner, and which cells of the (variable, fold) grid a score is defined on. Presence-absence with a joint multi-label head is what ships; an abundance or phenology response is a registration rather than a second fitting path.

Usage

```
register_response(name, spec, overwrite = FALSE)

responses()
```

Arguments

name	Name the response is asked for by.
spec	A list with elements prepare(y), returning the numeric matrix a learner is fitted on; activation, the name of the output transform ("sigmoid" or "identity"); loss, the name of the training objective ("binary_cross_entropy" or "squared_error"); metric, the default metric name; and cells(y, folds), returning the mask of scorable cells. Every learner that ships reads loss and activation from here: the encoders train under the loss and predict through the activation, and the

learners fitting one model per response take the family the loss names, logistic or Gaussian. The combiner minimises the same loss. An optional `weights(y, fitting)` returns a [unit, variable] matrix of case weights every learner fits under, with fitting a logical vector marking the rows the model is fitted on: whatever the head reads off the response, it reads off those rows, and it weights every row, so an encoder's inner validation loss is weighted as its fit is. The shipped head's is `positive_weights()`, and a head without one fits unweighted.

`overwrite` Replace an existing registration.

Value

The registered specification, invisibly.

Examples

```
responses()
```

roc_auc

The area under the ROC curve

Description

The whole curve rather than its best point. TSS is a maximum over cuts, so a small change in a prediction often moves it not at all and then moves it a long way; the area responds to every reordering, which is what makes it the steadier reading when many rescorings are compared, as in `occlusion()`. Tied predictions take the average rank.

Usage

```
roc_auc(y, p)
```

Arguments

`y` Observed presence-absence, 0/1 or logical.
`p` Predicted scores for the same units, in the same order. Higher means presence.

Value

One number, or NA where the cell defines none.

Examples

```
roc_auc(c(0, 0, 1, 1), c(0.1, 0.2, 0.8, 0.9))
```

scorable_cells	<i>Which cells a score is defined on</i>
----------------	--

Description

A per-variable score needs both classes among the held-out units, and a per-variable model needs both classes among the units it was fitted on, so a (variable, fold) cell where either side of the split is one-class carries no score. The mask says which cells those are.

Usage

```
scorable_cells(y, folds)
```

Arguments

y	The response: a matrix or data frame of units by variables, with unit identifiers in the row names or in a leading character or factor column.
folds	A fold map from <code>fold_map()</code> , or any named integer vector of the same shape.

Details

It is computed from the response and the fold map alone, with no model involved. Every learner in a ladder is then restricted to the same cells, so their means share one denominator and every paired difference runs on matched cells. Computing it from a model instead would let a joint multi-label learner, which emits a number for every cell whether or not it could be fitted per variable, be scored on cells its opponents were never fitted on.

Value

A data frame of one row per (variable, fold) cell, of class `timesift_cells`, with the counts on each side of the split and a scorable flag.

Examples

```
set.seed(1)
y <- matrix(rbinom(600, 1, 0.2), nrow = 100,
            dimnames = list(sprintf("p%03d", 1:100), paste0("sp", 1:6)))
cells <- scorable_cells(y, fold_map(y, v = 5))
cells
```

score_predictions	<i>Score held-out predictions on the cells the mask allows</i>
-------------------	--

Description

The scoring every arm of a ladder and every candidate of a run goes through, reachable on its own for a prediction matrix that came from somewhere else: a combination of arms, a model fitted outside the package, predictions read back from a file.

Usage

```
score_predictions(y, p, folds, cells = NULL, metric = "roc_auc")
```

Arguments

<code>y</code>	The response, as a matrix of units by variables.
<code>p</code>	Held-out predictions for the same units and variables.
<code>folds</code>	A <code>fold_map()</code> result, or one fold per unit.
<code>cells</code>	A <code>scorable_cells()</code> mask. Computed from <code>y</code> and <code>folds</code> when left unset.
<code>metric</code>	Name of a registered metric to read the cells by, or a function of (<code>y</code> , <code>p</code>).

Details

A cell is one response in one fold. Scoring only the cells the mask allows is what keeps two arms comparable, so the mask is computed from the response and the fold map alone and never from a model.

Value

A data frame of one row per variable and fold, carrying the score and whether the cell was scorable.

Examples

```
set.seed(1)
y <- matrix(rbinom(120, 1, 0.4), nrow = 30,
            dimnames = list(sprintf("p%02d", 1:30), paste0("sp", 1:4)))
p <- matrix(runif(120), nrow = 30, dimnames = dimnames(y))
head(score_predictions(y, p, fold_map(y, v = 3)))
```

select_grain	<i>Choose the grain inside the training data, and score the whole procedure</i>
--------------	---

Description

`grain_ladder()` fits every candidate against one fold map and reports the grid, so reading the best grain off it and quoting that grain's score quotes a number the held-out units helped choose. This does the choosing inside the training data instead. Within each outer fold the training units are split again, every candidate is fitted on part of them and scored on the rest, the best is refitted on the whole outer training set, and the outer test fold is predicted once. The estimate that comes back is therefore of the procedure including its choice of grain, which is what an ecologist applying it to a new site would run.

Usage

```
select_grain(
  x,
  y,
  learners,
  folds = NULL,
  inner = 5L,
  rule = c("argmax", "coarsest_adequate"),
  threshold = NULL,
  interval = c("variables", "nested_cv"),
  repeats = 1L,
  response = "presence_absence",
  metric = NULL,
  compare = NULL,
  control = train_control(),
  seed = 1L,
  verbose = TRUE
)

## S3 method for class 'timesift_selection'
summary(object, ...)
```

Arguments

x	A <code>grain_matrix()</code> result, a <code>timesift_set()</code> , or a named list of representations. Its names are the grains being chosen between.
y	The response for the same units.
learners	A learner, a list of them, or names of registered ones, as <code>grain_ladder()</code> takes. Named alongside the grains they form the candidate set.
folds	The outer fold map, from <code>fold_map()</code> or any named integer vector. Built with the defaults of <code>fold_map()</code> when not given.

inner	Number of inner folds the selection is made on, or a function of the outer training response returning a fold map for those units. A count deals the inner folds by the grouping the outer fold map carries, so what <code>grouped_cv()</code> kept whole outside stays whole inside.
rule	How a candidate is chosen from its inner scores. "argmax" takes the highest. "coarsest_adequate" takes the coarsest candidate whose inner score lies within one standard error of the highest, the one-standard-error rule of Breiman, Friedman, Olshen and Stone (1984) and of Hastie, Tibshirani and Friedman (2009, section 7.10) with coarseness in place of model complexity. See Choosing a candidate.
threshold	NULL, or the rule of <code>decision_threshold()</code> a presence-absence cut is learned by: "youden", the cut that maximises TSS, "kappa" or "prevalence". See A cut learned inside the training data.
interval	Which interval to report beside the across-variable one, which is always reported: "variables" for that one alone, or "nested_cv" for an interval for the procedure's risk. See What the interval is for.
repeats	Repetitions of the nested cross-validation, each on its own fold map. The first is the map the estimate was computed on.
response	Name of the registered response head.
metric	Name of a registered metric the selection is made on, or NULL for the response's own. The estimate is reported under every registered metric whichever this is.
compare	A <code>grain_ladder()</code> result on the same units, response and outer fold map, whose arms the selected procedure is contrasted against cell by cell. NULL for no contrast.
control	<code>train_control()</code> , the training settings every neural learner reads, in the inner search and in the refit alike. A learner carrying a control of its own overrides it on the settings that control names.
seed	Seed for the inner splits. Each outer fold splits under seed plus its own number, so no two outer folds inherit the same inner partition.
verbose	Report each outer fold and what it selected as it runs.
object	A selection.
...	Ignored.

Details

A candidate is a (grain, learner) pair: the grains are the elements of the representation set, which is where a grain and the statistic its grains are summarised by are both named, and the learners are the ones passed. Both are registry entries or objects built by `learner()`, so a new grain, a new grain summary or a new candidate model widens the search with no change here.

What the estimate is of: the expected held-out score of the whole pipeline, selection included, on units drawn as these were. What it is not: the score of the winning grain. That is higher, by the amount selection buys itself, and the difference between the two is the quantity this function exists to keep out of a reported number. It also does not say the selected grain is the one a mechanism acts at; it says that grain predicted best on the units the selector saw.

The cost is the ladder's, multiplied by the number of inner folds: $v_{\text{outer}} * (v_{\text{inner}} * \text{candidates} + 1)$ fits. With a neural learner that is where an overnight run goes.

Value

A `timesift_selection`: a list carrying selected, one row per outer fold with the candidate it chose, the inner score it chose on, the highest inner score in that fold (`inner_best`) and that score's standard error (`inner_se`); `estimate`, the nested score under every registered metric with its standard error across variables; `contrast`, one `paired_contrast()` row against each arm of compare, or NULL; `candidates`, the set that was searched; and `scores`, the per-cell rows of the selected procedure under the selection metric, in the layout `grain_ladder()` returns. The held-out prediction of every unit is in the `predictions` attribute and the scorable-cell mask in `cells`. `inner` holds every candidate's inner score and standard error in every outer fold. With `threshold` set, the `estimate` carries the score, its interval and the interval's name in `interval`, one row per metric and interval. With `interval = "nested_cv"` it also carries `nested_cv`, the same rows with the estimator's own quantities beside them, and `final`, the procedure fitted on every unit, whose risk that interval is for. With `threshold` set, the `estimate` carries one further row, `tss_inner_cut`, the procedure's TSS at the learned cuts; `thresholds` holds the cut of every outer fold and variable; and `cut_scores` the per-cell rows it is averaged from, in the layout of `scores`. Both are NULL otherwise.

Choosing a candidate

Inside each outer fold every candidate carries an inner score, the mean over variables of its per-variable mean over the inner folds, and a standard error, the standard deviation over the inner folds of the fold's own score (the mean over the variables scored in that fold) divided by the square root of the number of inner folds. `rule = "argmax"` takes the highest inner score, and on an exact tie the candidate declared first.

`rule = "coarsest_adequate"` first finds that highest score and its standard error, calls every candidate scoring at least the highest minus one standard error adequate, and takes the coarsest adequate one. Coarseness is read off the representation as the package holds it: fewer bins is coarser, and between two candidates with the same number of bins, fewer channels is coarser. A tie on both goes to the higher inner score, then to the candidate declared first. Where one candidate scores more than a standard error above every other, the two rules agree; where the inner profile is flat, this one returns the least storage the record can be kept at without a measured loss inside the training data. A standard error that cannot be computed, because a candidate was scored in fewer than two inner folds, is taken as zero, so the rule falls back to the candidates tied with the highest score.

What the interval is for

The across-variable interval, the one every level of the package reports, is the estimate plus or minus a Student's *t* quantile times the standard error across the response variables. Its spread is the spread of true skill between variables, and it cannot see the error every variable shares, since all of them are fitted and scored on the same units and the same folds. It is an interval over the variables of this dataset, and not an interval for what the procedure would score on a new sample.

`interval = "nested_cv"` adds one that is meant to be, by the nested cross-validation of Bates, Hastie and Tibshirani (2024). Inside every repetition, each outer training set is cross-validated again over the remaining folds of the same map, which gives the mean squared error of a cross-validation estimate as the difference of two terms it can estimate: the squared gap between the inner estimate and the held-out fold's score, less the variance of that fold's score. The paper's error is a mean of per-unit losses; here a fold's score is the mean over the variables scorable in it, the inner estimate is averaged as the reported estimate is, and the variance of a fold's score is its delete-one jackknife variance over the units of the fold, which for a mean of per-unit losses is exactly the

paper's $\text{var}(e) / |I_k|$. The centre is the nested estimate less the paper's bias correction, its Appendix C, so the interval is for the risk of the procedure fitted on a sample of this size, which is the fit final holds. The square root of the estimated mean squared error is held between the jackknife standard error of the estimate and the square root of the fold count times it, as the paper's section 4.3.2 has it.

The width departs from the paper in one respect. The paper's width is the mean squared error of the plain cross-validation estimate, and its bias correction is added to the centre as a shift with no spread of its own. The correction is $1 + (K - 2) / K$ times the gap between two cross-validation estimates on the same units, and in the package's benchmark (`inst/benchmark/`) that gap moved from sample to sample by more than the estimate it corrects where the sample was small or the signal absent, so the paper's interval covered a nominal 95% at 0.86 to 0.93 there at ten repetitions. The width here is therefore the same identity applied to the corrected estimator: inside every outer training set the nested cross-validation is run once more, over the unordered triples of folds, which gives the bias-corrected estimate that training set alone would report, and the squared gap between that and the held-out fold's score replaces the plain estimate's; the bounds are the corrected centre's own jackknife standard error, with every prediction held fixed, and the square root of the fold count times it. The paper's width is reported beside it as `se_bates` in `nested_cv`. On the package's cheap recovery design (`tests/testthat/test-interval.R`: 150 units, five outer folds, one repetition, 200 replicates) the interval covers a nominal 95% at 0.96 with a signal and 0.94 without, where the paper's covers 0.955 and 0.895.

The cost is the selection's, multiplied: one repetition fits the procedure once for every unordered pair and every unordered triple of outer folds, `choose(v_outer, 2) + choose(v_outer, 3)` fits, and every repetition after the first refits the outer folds as well. More repetitions steady the estimate of the mean squared error; the paper uses two hundred random splits, which is affordable where a fit is cheap and is not where a fit is a neural network.

A cut learned inside the training data

TSS read at the cut that maximises it on the scored units is biased upward, most where presences are few (`tss_inflation()`). With `threshold` set, each outer fold learns one cut per variable on the inner out-of-fold predictions of the candidate it selected, which the inner search has already made for every outer training unit, by `decision_threshold()` under the rule named. The cut is then frozen and the outer test fold's predictions are read at it by `tss()`. No unit of an outer test fold enters the cut its own fold is read at. The inner out-of-fold predictions come from models fitted on part of the outer training set and the held-out predictions from the refit on all of it, so the cut is learned on predictions of the same candidate from slightly smaller training sets.

References

Bates, S., Hastie, T. and Tibshirani, R. (2024). Cross-validation: what does it estimate and how well does it do it? *Journal of the American Statistical Association* **119**(546), 1434-1445. doi:10.1080/01621459.2023.2197686

See Also

`grain_ladder()` for the grid this selects from, and `paired_contrast()` for the comparison the contrast element holds.

Examples

```
set.seed(1)
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 200)
units <- sprintf("p%02d", 1:60)
warmth <- rnorm(60)
d <- data.frame(
  plot = rep(units, each = length(t)), t = rep(t, length(units)),
  temp = as.numeric(vapply(warmth, function(w) w + sin(seq_along(t) / 300) + rnorm(length(t)),
    numeric(length(t))))
y <- matrix(rbinom(120, 1, plogis(c(warmth, -warmth))), nrow = 60,
  dimnames = list(units, c("sp1", "sp2")))
x <- grain_matrix(d, plot, t, temp, grain = c("week", "month"))
sel <- select_grain(x, y, elasticnet(), folds = fold_map(y, v = 3), inner = 3,
  verbose = FALSE)

sel
sel$estimate
```

simulate_records	<i>Simulate sensor records whose response acts at a known temporal grain</i>
------------------	--

Description

Draws units carrying a year of sensor readings and a multi-variable presence-absence response whose dependence on the record is a fixed linear functional of the record at one named grain. The grain is therefore known before anything is fitted, which is what makes the output usable for asking whether `select_grain()` finds it.

Usage

```
simulate_records(
  n = 300L,
  mechanism = c("none", "event", "season", "lag"),
  variables = 10L,
  prevalence = 0.1,
  auc = 0.75,
  from = "2021-09-01",
  days = 365L,
  step_hours = 3,
  seasonal = 8,
  offset_sd = 1,
  anomaly_sd = 1,
  anomaly_days = 2,
  offset_effect = 0,
  sensor_sd = 0.3,
  year_start = "09-01",
  seed = 1L,
```

```

    draw = 1L
  )

```

Arguments

n	Number of units to draw.
mechanism	Which generating mechanism, see The mechanisms.
variables	Number of response variables. Each gets its own weights within the mechanism.
prevalence	Marginal probability of presence, shared by every variable.
auc	Population area under the ROC curve of the driver against the response.
from	First reading instant, "YYYY-MM-DD", read as UTC.
days	Length of the record in days.
step_hours	Sampling step in hours. Must divide 24.
seasonal	Amplitude of the seasonal cycle shared by every unit, in reading units.
offset_sd	Standard deviation of the unit-level thermal offset.
anomaly_sd	Marginal standard deviation of each unit's AR(1) anomaly.
anomaly_days	Correlation time of that anomaly, in days.
offset_effect	Weight the unit-level offset enters the driver with. At the default 0 the driver reads the unit's <i>anomaly</i> alone, so a grain coarse enough to average the anomaly away loses the signal. At 1 the offset carries the response as well, and since every grain however coarse reports the offset, every grain is then equally good: that is the grain-invariant control, not a temporal mechanism.
sensor_sd	Standard deviation of the measurement noise added to the latent record. The response is generated from the latent record; the readings returned carry this noise.
year_start	"MM-DD" boundary of the hydrological year, passed to <code>grain_matrix()</code> when the mechanism's bins are located. Use the same value when representing the readings.
seed	Seed of the design: the weights and the link coefficients. Two calls with the same seed and different draw share a design and draw independent units, which is what lets a held-out deployment sample be drawn from the same population as a training sample.
draw	Seed of the unit draw. Also names the units, so two draws never collide.

Value

A `timesift_simulation`: a list with readings, the long table `grain_matrix()` takes; `y`, the [unit, variable] 0/1 response; `driver`, the standardised driver `z` behind it; `grain`, the true grain or NA; `weights`, the [reading, variable] weights defining the driver; `link`, the solved `b0` and `b1`; and `design`, the settings the draw is reproducible from.

What the true grain is

The response is driven by $g_{ij} = \sum_t w_j(t) * a_i(t)$, a weighted mean of unit i 's latent *anomaly*: the record with the seasonal cycle every unit shares and the unit's own constant offset taken out, since neither of those is temporally located and a grain of any width reports both. The weights w_j are constant within the bins of one grain and zero outside a short stretch of them, so g is exactly a linear combination of that grain's bin means. The true grain of a mechanism is the **coarsest grain of `grain_matrix()` at which g is still an exact linear functional of the representation**: at that grain and at every grain whose bins nest inside it, no information about g has been averaged away, and at any coarser grain some has. Finer grains keep the information but spread it over more coefficients, so they lose to the true grain by variance rather than by bias, which is the tension the selection has to resolve.

The mechanisms

"none" No temporal signal. The driver is a standard normal drawn independently of the record, so nothing in the readings carries information about the response and no grain is correct. grain is NA.

"event" An isolated event. The weights are uniform over three consecutive **day** bins at a fixed calendar position, one position per variable. True grain "day": a week bin mixes the three days with four others and cannot be unmixed.

"season" A smooth seasonal response. The weights are uniform over one whole **season** bin, one season per variable, cycled. True grain "season": month and day bins nest inside a season so they are exact too, a year bin mixes all four seasons.

"lag" A lagged, cumulative response. The weights decay geometrically over four consecutive **week** bins from a fixed anchor, one anchor per variable. True grain "week": day bins nest inside weeks so they are exact too, month bins straddle week boundaries.

How the skill is set rather than emergent

The driver is standardised to a standard normal by its population mean and standard deviation, both computed in closed form from the generating parameters rather than from the drawn units, so every draw and every chunk of a draw is on the same scale. The response is $y_{ij} \sim \text{Bernoulli}(\text{plogis}(b_0 + b_1 z_{ij}))$ with b_0 and b_1 solved by numerical integration so that the marginal prevalence is prevalence and the population area under the ROC curve of z is auc. auc is therefore a ceiling no fitted model reaches: the response is generated from the latent record and the readings carry sensor_sd of measurement noise on top of it, and the weights have to be estimated.

See Also

`select_grain()`, which this exists to test, and `grain_matrix()`, whose calendar the weights are defined on.

Examples

```
sim <- simulate_records(n = 40L, mechanism = "event", variables = 2L, days = 60L)
sim
sim$grain
x <- grain_matrix(sim$readings, unit, time, reading, grain = c("day", "month"))
```

```
dim(x$day)
```

stepwise

Forward selection by AIC on the flattened representation

Description

One generalised linear model per variable, its predictors chosen by forward selection over every bin-by-channel column, admitting a column while it lowers AIC and stopping at a fixed budget. Each candidate enters as an orthogonal polynomial, so a term can be non-monotone in the reading the way a niche optimum is. The family is the response head's: logistic under a binary cross-entropy loss, Gaussian under a squared-error one. So are the case weights, so a rare response weighs here what it weighs in every other learner.

Usage

```
stepwise(data = NULL, max_terms = 3L, degree = 2L)
```

Arguments

data	A representation the learner is pinned to, or NULL to run across every representation of the run.
max_terms	Predictors admitted before selection stops.
degree	Polynomial degree each admitted column enters at.

Details

Selection happens inside whichever units the learner is handed, so under [grain_ladder\(\)](#) it is redone in every fold. That is the footing the other learners are fitted on. Reported beside a penalised fit it also prices discrete selection: choosing a handful of columns out of hundreds is high variance, and that variance is a cost of the selector rather than of the features.

Value

A [learner\(\)](#).

Examples

```
stepwise(max_terms = 3)
```

timesift

*Fit and compare representations of time-varying data***Description**

One call from two tables to a scored comparison and a held-out estimate of choosing among it. `targets` is one row per thing to predict and `series` is the long, time-stamped record belonging to those rows. Every representation in `sift` is built and every learner in `models` is paired with the ones it can read; each pair is a candidate.

Usage

```
timesift(
  targets,
  series = NULL,
  y,
  x = NULL,
  id = NULL,
  time = NULL,
  target_time = NULL,
  static = NULL,
  models = NULL,
  sift = NULL,
  ensemble = TRUE,
  resampling = cv(),
  inner = 5L,
  rule = c("argmax", "coarsest_adequate"),
  response = "presence_absence",
  metric = NULL,
  control = train_control(),
  keep_fits = FALSE,
  seed = 1L,
  verbose = TRUE
)
```

Arguments

<code>targets</code>	A data frame, one row per prediction target.
<code>series</code>	A long data frame of readings, or <code>NULL</code> to fit on <code>static</code> alone.
<code>y</code>	Columns of <code>targets</code> holding the response, as a tidyselect expression such as <code>starts_with("sp_")</code> .
<code>x</code>	Columns of <code>series</code> holding the readings, as a tidyselect expression. Defaults to every numeric column but <code>id</code> and <code>time</code> .
<code>id</code>	Column naming the unit, present in both tables. A bare column name or a string.
<code>time</code>	Column of reading instants in <code>series</code> , <code>POSIXct</code> .

target_time	Column of targets anchoring each row in time, POSIXct. Optional, and what a unit carrying several targets through time needs.
static	Columns of targets carried alongside the representation, as a tidyselect expression. None by default.
models	A learner, a set of them from <code>c()</code> , or a list. Defaults to <code>elasticnet()</code> .
sift	The representations a learner without a data = of its own is run across. A <code>grains()</code> or <code>lookbacks()</code> set, a set from <code>c()</code> , a bare vector of grain names, a single representation, or a list of them. Defaults to <code>grains("auto")</code> .
ensemble	TRUE for the default stack, FALSE for none, or an <code>ensemble()</code> spec.
resampling	The outer split: <code>cv()</code> , <code>grouped_cv()</code> , a fold vector, or a <code>fold_map()</code> result.
inner	Number of inner folds the choice and the stack's weights are made on inside each outer training set, a function of the outer training response returning a fold map for those targets, or NULL to compare the candidates on the outer folds without an estimate. A count deals the inner folds by the grouping the outer split carries, as <code>select_grain()</code> does.
rule	How a candidate is chosen from its inner scores, "argmax" or "coarsest_adequate", as in <code>select_grain()</code> .
response	Name of the registered response head.
metric	Name of a registered metric, or a function of (y, p), or NULL for the response head's own. It is what the candidates are chosen on and what the report reads. Whichever it is, it travels with the fit and is what every later rescoring reads; a function is reported as <function>. The estimate is also reported under every registered metric.
control	<code>train_control()</code> , the training settings every neural learner reads.
keep_fits	Keep every per-fold fitted candidate beside the refits.
seed	Seed for the inner splits. Each outer fold splits under seed plus its position.
verbose	Report each outer fold as it runs.

Details

Within each outer fold of resampling the training targets are split again into inner folds. Every candidate is cross-validated on that inner split, the rule picks one on its inner score, and the stack's weights are fitted on the inner out-of-fold predictions. Every candidate is then refitted on the whole outer training set and predicts the outer test fold, and the selected candidate's prediction and the prediction combined under that fold's weights are kept. Nothing the outer test fold holds enters the choice or the weights it is scored under, so estimate is of the procedure, selection and stacking included, which is what an ecologist applying it to a new site would run.

The same refits give every candidate an out-of-fold prediction on the outer folds, and scores holds those. They say where predictive skill saturates as the record is read more coarsely, which is the measurement the package exists for, but the highest of them is a number the held-out targets helped choose: read the candidates for the shape of the comparison and estimate for the level. With inner = NULL no inner search is run, the candidates are compared on the outer folds alone and no estimate is made.

The cost is $v_outer * (v_inner + 1) * \text{candidates}$ fits for the evaluation and one refit per candidate on every target, against $v_outer * \text{candidates}$ for the comparison alone.

Value

A `timesift` object, a list carrying:

- `estimate`: the held-out score of the selected candidate (`arm = "selected"`) and of the stack (`arm = "ensemble"`), one row per metric, with the standard error and the 95% interval across response variables. An interval across the variables of this dataset, not one for a new sample: every variable is fitted and scored on the same targets and folds, so the error they share is not in it. NULL with `inner = NULL`.
- `selected`: one row per outer fold, the candidate it chose, the inner score it chose on, the highest inner score and that score's standard error; `inner`: every candidate's inner score in every outer fold; `fold_weights`: the stack's weights in every outer fold, one row per fold. All NULL with `inner = NULL`.
- `predictions`: the held-out prediction of every target under the selected candidate and under the stack.
- `candidates`, `scores` and `oof`: every candidate, its per-cell scores on the outer folds and its outer out-of-fold predictions.
- `choice`, `models`, `stack` and `weights`: the procedure applied to every target, which is what `predict()` uses. Every candidate is refitted on all of them; `choice` is the candidate the rule takes on the outer scores, with the outer folds as the split it chooses on, and `stack` holds weights fitted on the outer out-of-fold predictions.
- `representations`, `fits`, `folds`, `cells`, `y`, and the `metric`, `response`, `spec` and `call` it was asked for.

Rules the entry point enforces

`static` is never implicit: a column of targets that is neither the response, the identifier nor the anchor is ignored unless `static` names it, because a predictor nobody asked for is worse than one that is missing.

One target row per id, unless `target_time` says where in time each row sits. Repeated identifiers without an anchor are an error naming them.

With `target_time`, every representation has to be anchored on the target, so `native()`, `grain()` and `multigrain()` are refused and `sift` must be given as `lookbacks()`. There is no default set of spans, because there is no defensible one.

Without series, `static` is the whole predictor block and `sift` is ignored.

What a learner may be handed

A learner declares whether it reads a tabular block or a sequence. A tabular learner given `native()` is refused before anything is built, and a sequence learner given a representation of one bin is refused once the array says how many bins it has. Inside a `sift` expansion such a pair is skipped and reported once by name; named explicitly through a learner's `data` = it is an error.

See Also

`build_representation()` for the array a candidate reads, `fold_map()` for the splits.

Examples

```
set.seed(1)
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 90)
units <- sprintf("p%02d", 1:30)
warmth <- rnorm(30)
logger <- data.frame(
  plot = rep(units, each = length(t)), datetime = rep(t, 30),
  temp = as.numeric(vapply(warmth, function(w) w + sin(seq_along(t) / 300), numeric(length(t))))
)
plots <- data.frame(plot = units,
  sp_a = rbinom(30, 1, plogis(2 * warmth)),
  sp_b = rbinom(30, 1, plogis(-2 * warmth)))

fit <- timesift(plots, logger, y = starts_with("sp_"), id = plot, time = datetime,
  sift = grains("week", "month"), resampling = cv(v = 3L),
  ensemble = FALSE, verbose = FALSE)

fit
```

timesift_report

What a run found

Description

Two kinds of row. A candidate row is the candidate's mean score on the outer folds, how many responses it scored highest on, and whether one fitted model covered those responses or one was fitted per response. These rows are the comparison: they share folds and cells, so their shape is read across grains and learners, but the highest of them was picked out on the folds it is scored on. The selected and ensemble rows are the procedure's held-out score, the choice and the weights made inside every outer training fold, and they are the level to quote.

Usage

```
## S3 method for class 'timesift'
summary(object, ...)

## S3 method for class 'timesift'
print(x, ...)

## S3 method for class 'timesift_summary'
print(x, ...)
```

Arguments

object	A timesift result.
...	Ignored, so that the methods take the arguments their generics declare.
x	A timesift result, or the table this returns.

Details

Both columns beside a candidate's mean are worth reading. A candidate can carry the ensemble without winning a single response, which is what won shows and a mean alone hides; and a joint model and a per-response one reach the same [target, response] matrix by different routes, which is what responses records.

A candidate the run built no representation for, because its learner cannot read the representation it was paired with, is listed with no mean rather than dropped, so the report says what was asked for as well as what ran.

Value

A data frame of class `timesift_summary`, one row per candidate and, where the run made an estimate, one for the selected candidate and one for the stack. It carries the mean score, its standard error across responses for the two procedure rows, the responses won, how the responses were covered, and scored: "outer folds" for a candidate, "nested" for the procedure. The weights fitted on every target are in the `weights` attribute and the candidate chosen on every target in `choice`.

<code>timesift_set</code>	<i>Several built representations of the same targets</i>
---------------------------	--

Description

A named list of arrays covering the same targets and differing only in how the record was reduced: several calendar grains, several lookback spans, a block of features beside a sequence. It is keyed by the label each representation is reported under, which is what `grain_ladder()` fits across and what a candidate in a `timesift()` fit is named by.

Usage

```
timesift_set(x)
```

Arguments

<code>x</code>	A named list of built representations – <code>grain_matrix()</code> , <code>lookback_matrix()</code> or <code>feature_matrix()</code> results – or a single one.
----------------	--

Value

A `timesift_set`: the list, keyed by representation label.

Examples

```
t <- seq(as.POSIXct("2021-09-01", tz = "UTC"), by = "hour", length.out = 24 * 60)
d <- data.frame(plot = rep(c("a", "b"), each = length(t)), t = rep(t, 2),
               temp = rnorm(2 * length(t)))
s <- grain_matrix(d, plot, t, temp, grain = c("day", "week", "month"))
s
names(s)
```

torch_learners

*Sequence encoders with a joint multi-label head***Description**

Three encoders, all fitted the same way and all ending the same way: one encoder maps a unit's representation to an embedding and a single linear layer maps that embedding to one output per variable, so every variable is predicted together from a shared representation. Pooling strength across variables is what makes the rarer ones learnable at all at these sample sizes.

Usage

```
mlp(data = NULL, hidden = c(512L, 256L), dropout = 0.3, ...)
```

```
cnn(
  data = NULL,
  channels = c(16L, 32L, 64L, 128L),
  kernel = 7L,
  dropout = 0.3,
  ...
)
```

```
rescnn(
  data = NULL,
  channels = c(32L, 64L, 128L, 256L),
  blocks_per_stage = 2L,
  kernel = 7L,
  dilations = c(1L, 2L, 4L, 8L),
  dropout = 0.3,
  ...
)
```

Arguments

data	A representation the learner is pinned to, or NULL to run across every representation of the run.
hidden	Hidden layer widths, for the fully connected encoder.
dropout	Dropout rate.
...	Training settings for this learner, named as in train_control() .
channels	Channel width of each stage.
kernel	Convolution kernel width.
blocks_per_stage	Residual blocks in each stage.
dilations	Dilation of each stage, cycled if shorter than channels.

Details

`m1p()` flattens the channels and builds in no temporal geometry. It is what separates the effect of the model class from the effect of the representation: where it matches a penalised regression, the difference a convolutional encoder makes is the convolution rather than the network.

`cnn()` is blocks of a one-dimensional convolution, batch normalisation, a rectified linear activation and max pooling, then global average pooling. Pooling becomes an identity once a sequence is shorter than its kernel, so the same stack runs at every grain of a ladder, including one bin per year.

`rescnn()` adds dilated residual blocks with squeeze-excitation channel gates, whose stage dilations widen the receptive field toward the seasonal scale without widening the kernel, and concatenates global average with global maximum pooling so extremes reach the head beside the level.

The three constructors carry architecture. How that architecture is trained is `train_control()`, which the run supplies; a setting named in `...` here overrides the run's control for this learner alone. What the head is trained toward is the response head's: its loss is the training objective and its activation the output transform, so a head registered with a squared-error loss and an identity activation trains the same encoders on a continuous response.

Every channel is standardised by its own centre and scale, computed over every unit and bin of the fitting units, so a static predictor appended as a channel sits on the same footing as a reading whatever its units are. The channels of `calendar_channels()` are the exception and are read at their own amplitude: they lie on the unit circle already, and being identical across units they reach a network as a bias, whose step under an adaptive optimiser grows with the scale of the input carrying it. Standardised, the four calendar channels of an hourly record move the fully connected encoder's first layer 1.4 times as far per step, and on the Schrankogel record that costs it 0.044 TSS (`inst/reproduce/README.md`).

A fitted encoder holds its weights as plain arrays and rebuilds the network when it predicts, so a fit saved with `saveRDS()` predicts after `readRDS()` in a fresh session.

Value

A `learner()`.

Examples

```
cnn(epochs = 5)
```

train_control	<i>Training settings every neural learner reads</i>
---------------	---

Description

The one place a training setting is defaulted. An architecture constructor carries its architecture and nothing else, the control carries how that architecture is trained, and both a whole run and a single learner take one, so there is never a second table of defaults to keep in step with this one.

Usage

```

train_control(
    epochs = 60L,
    batch_size = 64L,
    learning_rate = 0.001,
    weight_decay = 1e-04,
    early_stopping = 10L,
    val_frac = 0,
    device = "auto",
    seed = 1L,
    swa = FALSE,
    swa_start = 0.7
)

```

Arguments

epochs	Epoch budget the cosine schedule anneals over.
batch_size	Most targets per optimiser step. The fitting targets are cut into as few batches of at most this many as they divide into, of as equal a length as they can be, so no batch is a remainder of one.
learning_rate	Learning rate.
weight_decay	AdamW weight decay.
early_stopping	Epochs without an inner-validation improvement before training stops. Read only where <code>val_frac</code> holds a validation set back. Inf trains the whole budget and still restores the epoch with the lowest validation loss.
val_frac	Share of the fitting targets held back as an inner validation set, used for early stopping and for nothing else. It is never scored as a result. The set is drawn from every fit alike by a plain random permutation, so the fit on all targets that a run ends with also trains on the rest. At the default of 0 nothing is held back: every fitting target is trained on, the whole budget runs, and the fit keeps the last epoch, where the cosine schedule has annealed the learning rate to zero. On the Schrankogel weekly arm that epoch scores 0.007 AUC above the epoch early stopping keeps on a 15 percent split, and within 0.001 of the best epoch read on the test folds.
device	"auto" to take a graphics processor where there is one, NVIDIA's or Apple's, or a device name such as "cuda", "mps" or "cpu". A fitted encoder carries the setting rather than the device it resolved to, so a fit made on one machine predicts on another.
seed	Seed for initialisation, batching and the inner validation split.
swa	Average the weights of the tail epochs instead of keeping a single epoch. The schedule anneals to <code>swa_start</code> of the epoch budget and is then held flat while the remaining epochs' weights are averaged, and the batch-normalisation statistics are recomputed for the average. Early stopping is off while an average is being accumulated, so the averaging grain always runs.
swa_start	Share of the epoch budget after which averaging begins.

Details

A control records which of its settings were named in the call. Merging two controls therefore moves only the settings that were asked for: a learner given `train_control(epochs = 200)` reads 200 epochs and takes every other setting from the control the run was given.

Value

A `timesift_control`.

Examples

```
train_control()
train_control(epochs = 200L, device = "cpu")
```

tss	<i>The true skill statistic</i>
-----	---------------------------------

Description

Sensitivity plus specificity minus one, at the threshold that maximises it. This is the metric species distribution modelling reports. The shipped presence-absence response is scored by `roc_auc()` instead, which chooses no threshold, and TSS is reported beside it.

Usage

```
tss(y, p, threshold = NULL)
```

Arguments

y	Observed presence-absence, 0/1 or logical.
p	Predicted scores for the same units, in the same order. Higher means presence.
threshold	NULL for the maximum over every cut, or one cut, presence being predicted at $p \geq \text{threshold}$.

Details

A cut may only fall between distinct predictions: units sharing a prediction are decided together, so the same score comes back whatever order they arrived in. A cell holding only presences or only absences has no skill to measure and returns NA rather than a number.

The threshold is chosen on the same units the score is then read on, which is how the metric is defined in the literature and how it is defined here, and it inflates the level where presences are thin. `tss_inflation()` measures that inflation for a given design. How large it is depends on how a model's predictions are distributed as well as on the design, so two models of equal skill scored on the same cells can carry different inflations, and a paired difference in TSS is not free of it. Given a threshold learned elsewhere, the score is read at that cut instead, which is what `select_grain()` reports under `threshold =` with a cut learned on the inner folds.

Value

One number, or NA where the cell defines none.

Examples

```
tss(c(0, 0, 1, 1), c(0.1, 0.2, 0.8, 0.9))
tss(c(0, 0, 1, 1), c(0.9, 0.8, 0.2, 0.1))
tss(c(0, 0, 0, 0), c(0.1, 0.2, 0.8, 0.9))
tss(c(0, 0, 1, 1), c(0.1, 0.6, 0.8, 0.9), threshold = 0.5)
```

tss_inflation	<i>How much a self-selected threshold inflates the reported level</i>
---------------	---

Description

The true skill statistic is read at the threshold that maximises it, chosen on the same held-out units the score is then read on. That selection inflates the level, and by more the fewer presences a cell holds. Most code carries the inflation silently; this measures it for the presence counts of a given design.

Usage

```
tss_inflation(y, folds, skill = c(0.6, 0.7, 0.9), replicates = 200L, seed = 1L)
```

Arguments

y	The response: a matrix or data frame of units by variables, with unit identifiers in the row names or in a leading character or factor column.
folds	A fold map from <code>fold_map()</code> , or any named integer vector of the same shape.
skill	Population skill values to plant.
replicates	Replicates per value.
seed	Random seed.

Details

Predictions are simulated under a normal model in which the population skill is exactly `skill`, at the cell sizes and presence counts of the response and fold map supplied, and the level is read back exactly as `grain_ladder()` reports it. The gap between what comes back and the truth planted is the inflation.

The inflation is an expectation. A level read on one design is optimistic on average, not a bound every reading sits above: a single level can fall below the population skill. The model planted here is one distribution of predictions, and another at the same skill inflates by a different amount, which is also why a paired difference `paired_contrast()` takes is not free of it.

Value

A data frame with one row per planted value: the truth, the mean level read back, the inflation, and its interval across replicates.

Examples

```
set.seed(1)
y <- matrix(rbinom(1200, 1, 0.15), nrow = 200,
            dimnames = list(sprintf("p%03d", 1:200), paste0("sp", 1:6)))
tss_inflation(y, fold_map(y, v = 5), skill = c(0.6, 0.9), replicates = 40)
```

Index

artifacts, 3
as_sift (grains), 20
average_precision, 4

bind_channels, 5
bind_channels(), 7
build_representation, 6
build_representation(), 58

c(), 23, 57
c.timesift_learner (combine), 7
c.timesift_models (combine), 7
c.timesift_representation (combine), 7
c.timesift_sift (combine), 7
calendar_channels, 6
calendar_channels(), 36, 62
cnn (torch_learners), 61
combine, 7
coverage, 8
coverage(), 26
cv, 9
cv(), 57

decision_threshold (kappa_score), 29
decision_threshold(), 49, 51
digest_array, 10

elasticnet, 11
elasticnet(), 57
ensemble, 13
ensemble(), 15, 57
ensemble_combine, 14
ensemble_fit, 15
ensemble_fit(), 13, 14
ensemble_weights, 16

feature_matrix, 16
feature_matrix(), 60
fit_learner, 17
fold_map, 18
fold_map(), 3, 10, 23, 28, 46–48, 57, 58, 65

forest, 19

grain (native), 34
grain(), 6, 58
grain_contrasts, 21
grain_ladder, 22
grain_ladder(), 16, 22, 36, 37, 40, 43, 48–51, 55, 60, 65
grain_matrix, 24
grain_matrix(), 7, 8, 10, 18, 21, 23, 32–34, 48, 53, 54, 60
grains, 20
grains(), 34, 57
graphics::plot(), 39, 40
grouped_cv (cv), 9
grouped_cv(), 18, 49, 57

implied_skill, 27

kappa_score, 29

learner, 30
learner(), 13, 18, 20, 43, 49, 55, 62
learners (register_learner), 43
learners(), 43
lookback (native), 34
lookback(), 6
lookback_matrix, 31
lookback_matrix(), 7, 34, 60
lookbacks (grains), 20
lookbacks(), 57, 58

metrics (register_metric), 43
mlp (torch_learners), 61
model_agreement (kappa_score), 29
multigrain (native), 34
multigrain(), 6, 58

native, 34
native(), 6, 58

occlusion, 35
occlusion(), 23, 45

paired_contrast, 37
paired_contrast(), 22–24, 50, 51, 65
plot.timesift, 39
plot.timesift_ladder, 39
plot.timesift_selection, 40
positive_weights, 41
positive_weights(), 12, 20, 45
predict(), 58
predict.timesift, 42
predict.timesift(), 6, 34
predict.timesift_fit(fit_learner), 17
print.timesift(timesift_report), 59
print.timesift_summary
 (timesift_report), 59

read_cells(artifacts), 3
read_folds(artifacts), 3
read_response(artifacts), 3
readRDS(), 62
register_learner, 43
register_metric, 43
register_response, 44
rescnn(torch_learners), 61
responses(register_response), 44
roc_auc, 45
roc_auc(), 4, 38, 64

saveRDS(), 62
scorable_cells, 46
scorable_cells(), 4, 15, 47
score_predictions, 47
select_grain, 48
select_grain(), 23, 38, 40, 52, 54, 57, 64
simulate_records, 52
stats::predict(), 18
stepwise, 55
summary(), 36
summary.timesift(timesift_report), 59
summary.timesift_ladder(grain_ladder),
 22
summary.timesift_selection
 (select_grain), 48

timesift, 56
timesift(), 6, 9, 15, 36, 42, 60
timesift_report, 59
timesift_set, 60
timesift_set(), 9, 23, 25, 26, 48
torch_learners, 7, 61
train_control, 62
train_control(), 18, 23, 30, 49, 57, 61, 62
tss, 64
tss(), 29, 51
tss_inflation, 65
tss_inflation(), 27, 28, 38, 51, 64

write_cells(artifacts), 3
write_folds(artifacts), 3
write_response(artifacts), 3